CS 5402: Introduction to Data Mining

SP2026

(Lecture - 11)

Clustering

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

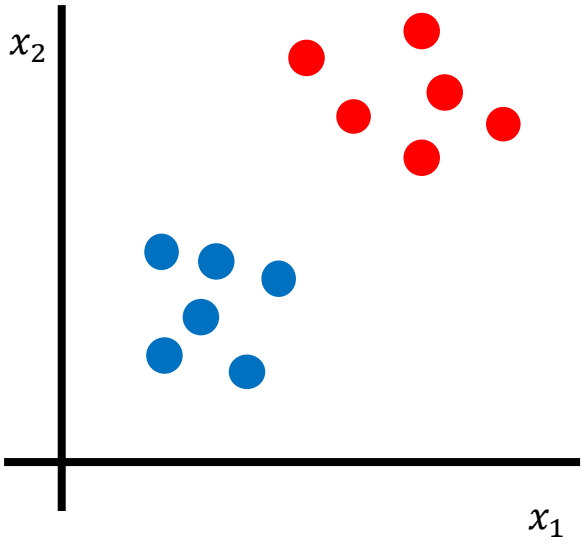
https://hyang428.github.io

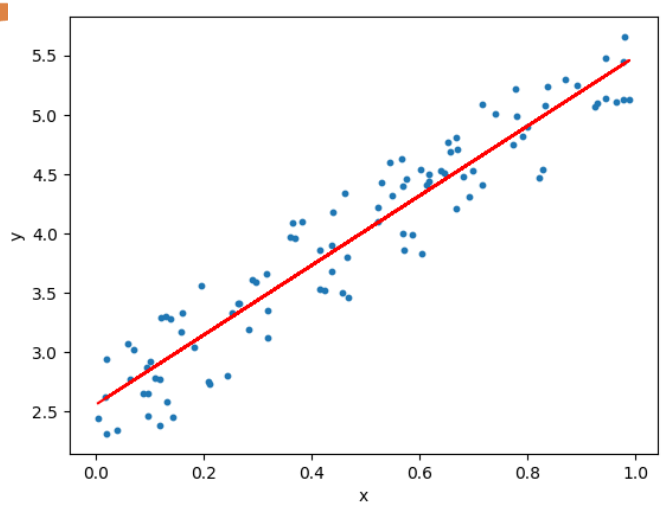
MISSOURI S&T

Outline

- What is clustering and its applications
- Clustering: Basics
- K-means clustering
- Expectation–Maximization (EM)
- Gaussian Mixture Models

Supervised Learning





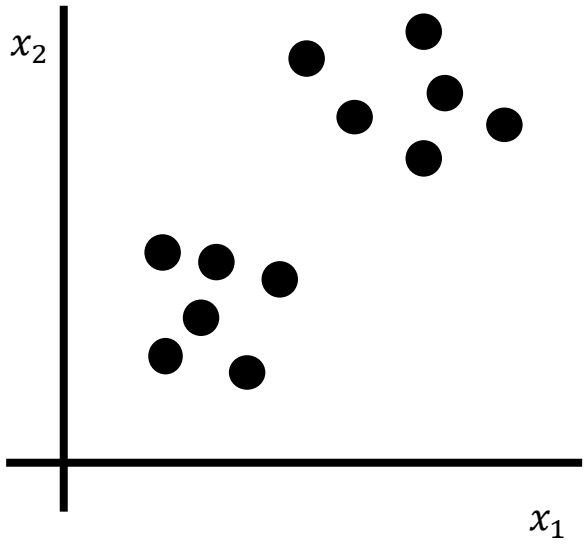
Training set:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$$

Predict target y given features x

Unsupervised Learning

Finding Patterns in the Unknown.



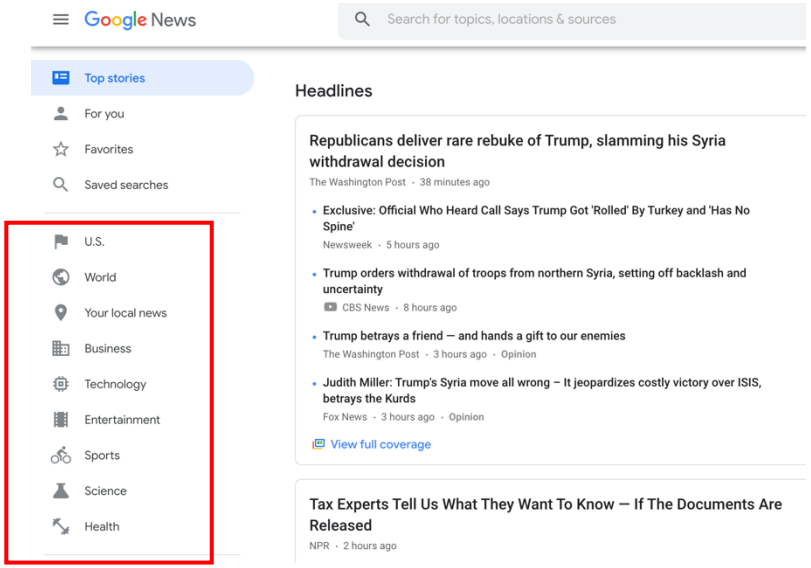
Training set: $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$

Unsupervised learning:

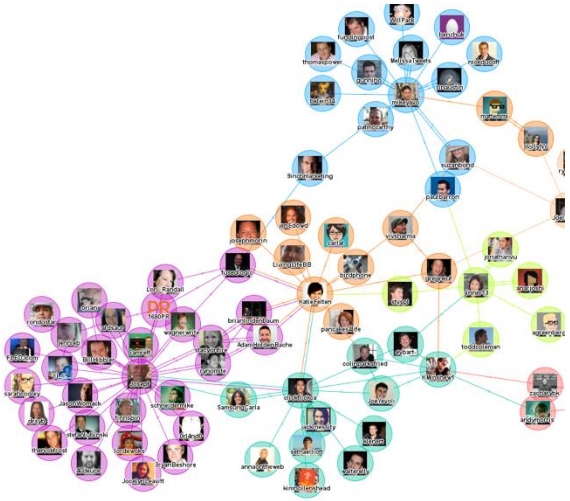
- Understand patterns of data (just "x")
- Useful for many reasons
  - Data mining ("explain")
  - Representation (feature generation or selection)
  - Missing data values ("impute")

One example: clustering

Real-World Examples:



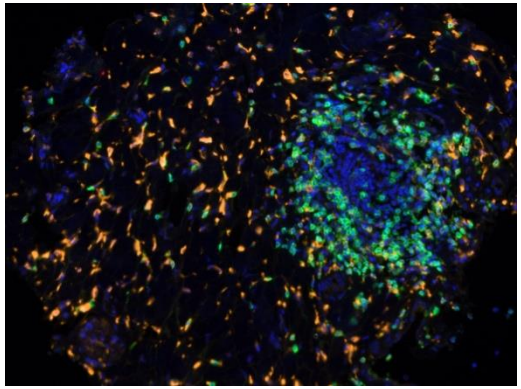
Newsfeed categorization



Social network analysis

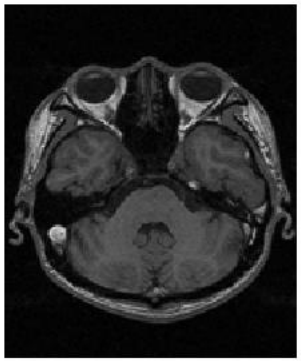
Images from www.news.google.com **and** https://www.touchgraph.com/news

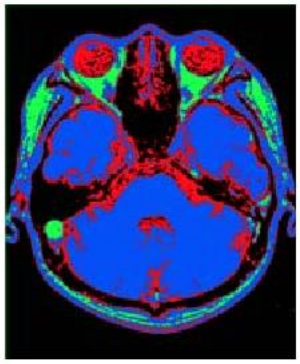
Real-World Examples:



Classifying unknown cell subpopulations

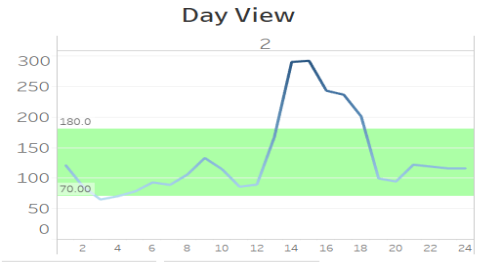
Images courtesy Dr. Jenny Lazrus, images.google.com, and Anet Sanchez





Medical image segmentation

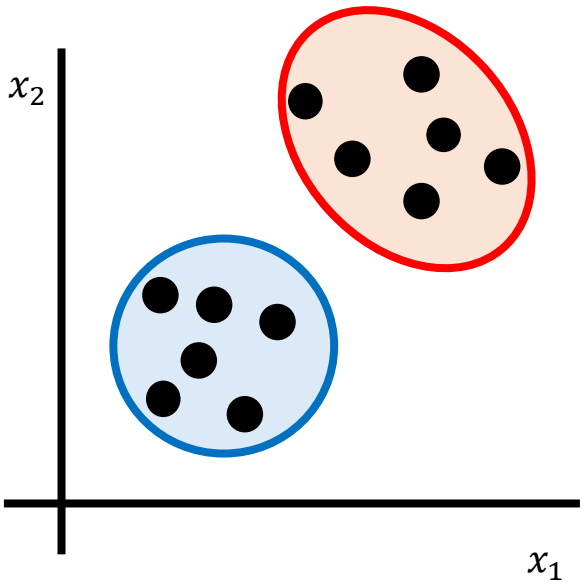




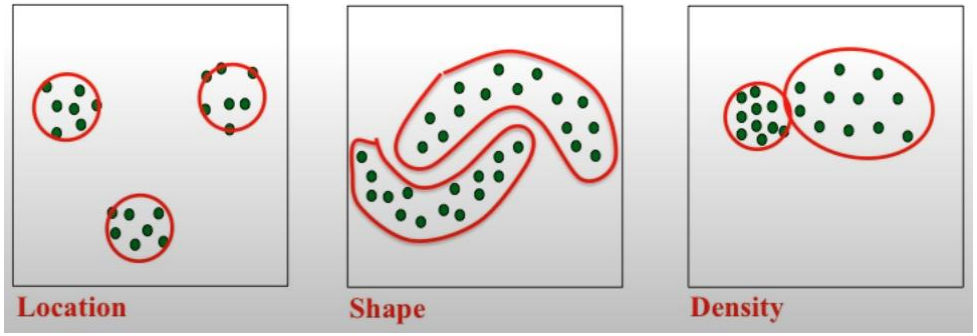
Detecting clinically important patterns in glucose data of diabetes patients

Clustering

Clustering is the task of automatically grouping similar data points together.



- Clustering describes data by "group"
- The meaning of "group" may vary by data!



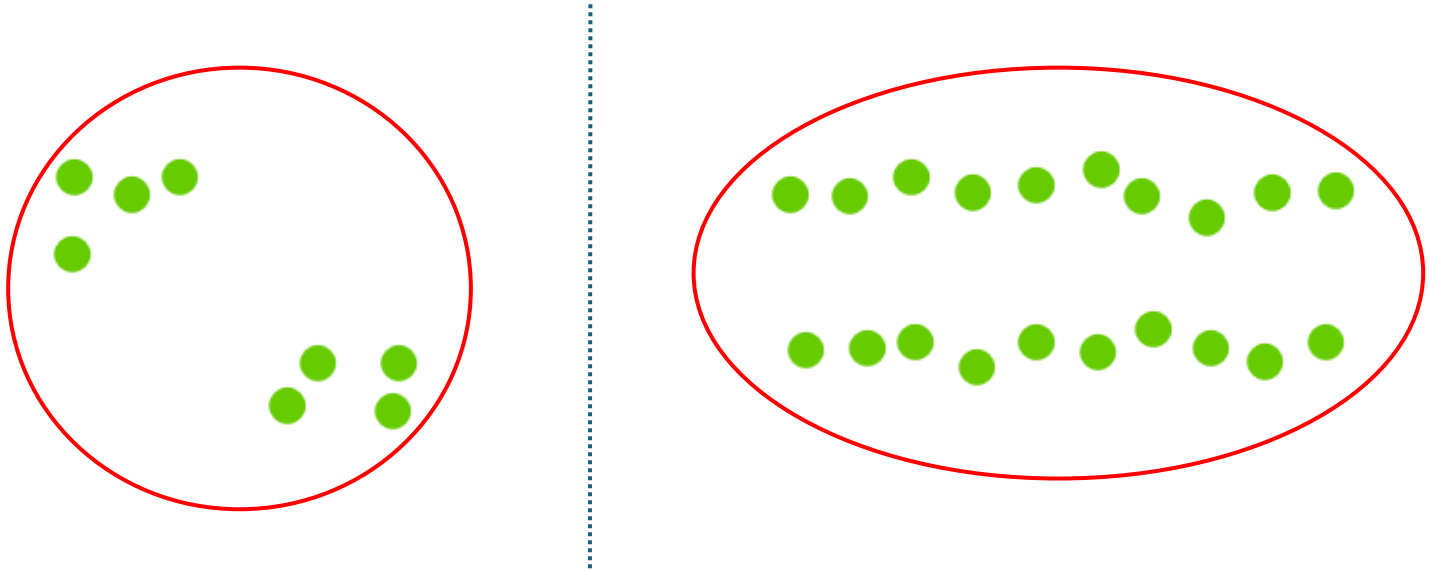
Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns



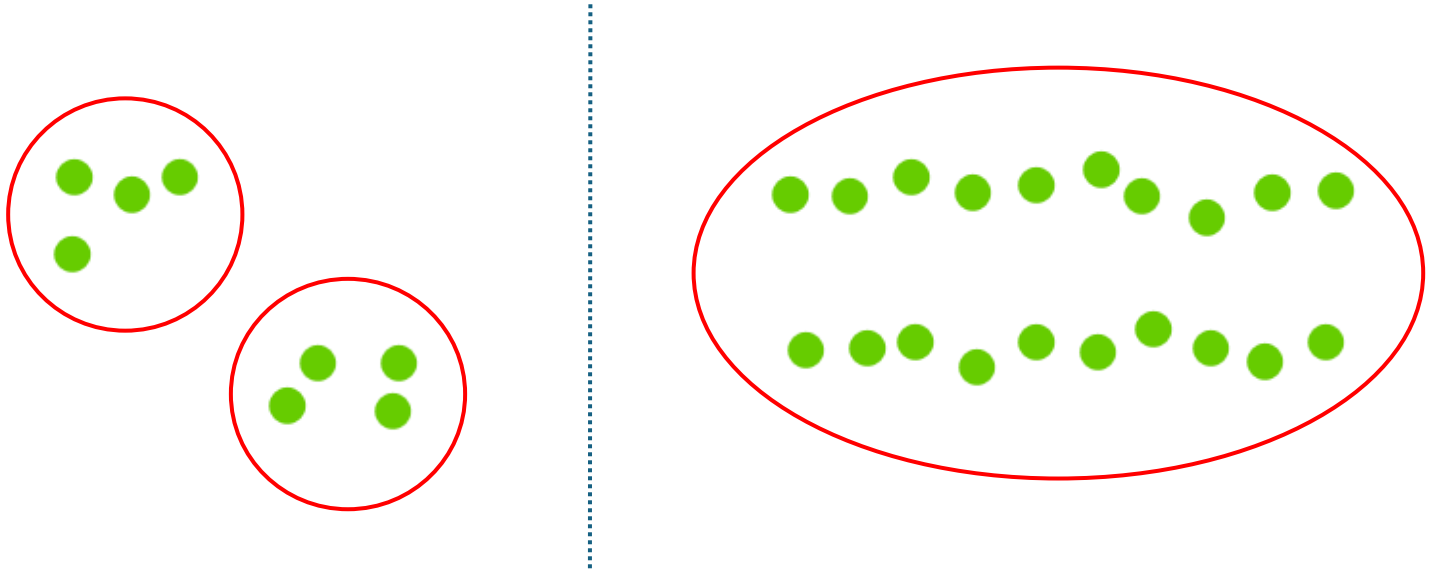
Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns



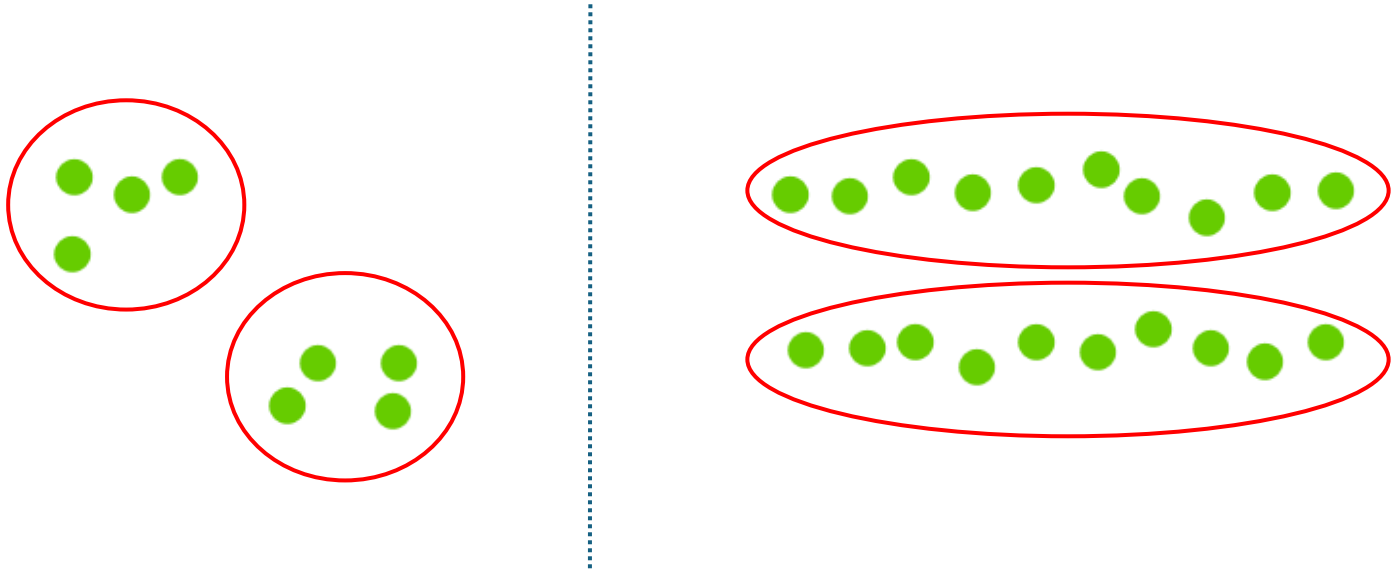
Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns



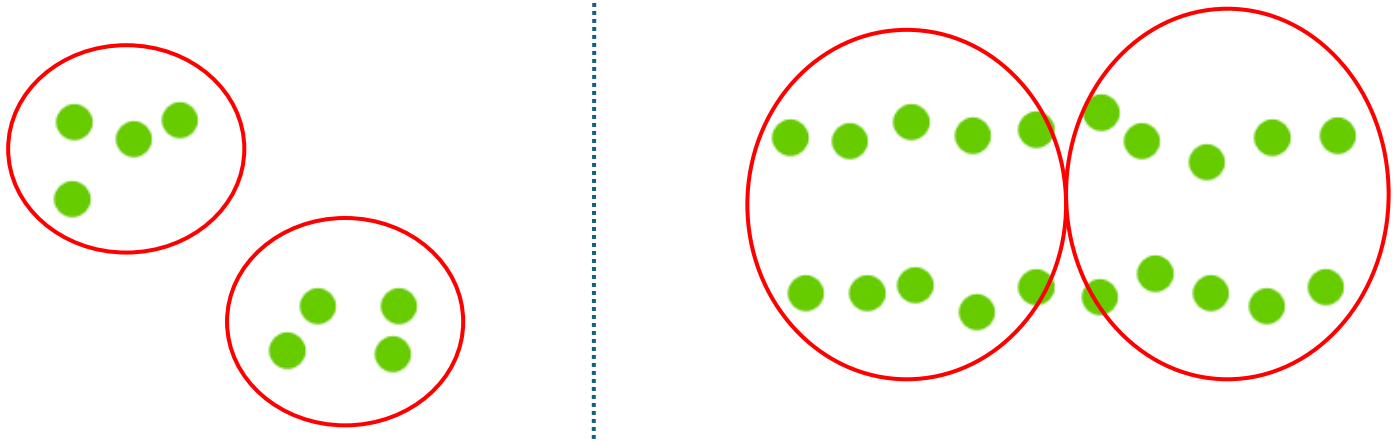
Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns



Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns



I. How many clusters?

II. What could "similar" mean?
   - One option: small Euclidean distance (squared)
   - Clustering results are crucially dependent on the measure of similarity (or distance) between "points" to be clustered.

Clustering: Basics

Distance (dissimilarity) measures: given sample set $X = \{x_1, x_2, \dots, x_n\}, x_i \in \mathcal{R}^m$. $x_i, x_j \in X, x_i = (x_{i,1}, x_{i,2}, \dots x_{i,m})^T, x_j = (x_{j,1}, x_{j,2}, \dots, x_{j,m})^T$, then the distance between x_i and x_j is defined as:

- Minkowski distance:

$$d(x_i, x_j) = \left(\sum_{k=1}^{m} |x_{i,k} - x_{j,k}|^p\right)^{\frac{1}{p}}, \text{ where } p \geq 1$$

  - When $p = 2$, Euclidean distance:

$$d(x_i, x_j) = \left(\sum_{k=1}^{m} |x_{i,k} - x_{j,k}|^2\right)^{\frac{1}{2}}$$

  - When $p = 1$, Manhattan distance:

$$d(x_i, x_j) = \sum_{k=1}^{m} |x_{i,k} - x_{j,k}|$$

  - When $p = \infty$, Chebyshev distance:

$$d(x_i, x_j) = \max_k |x_{i,k} - x_{j,k}|$$

Higher value, less similarity

Clustering: Basics

- Distance (dissimilarity) measures: given sample set $X = \{x_1, x_2, \dots, x_n\}, x_i \in \mathcal{R}^m$. $x_i, x_j \in X, x_i = \left(x_{i,1}, x_{i,2}, \dots x_{i,m}\right)^T, x_j = \left(x_{j,1}, x_{j,2}, \dots, x_{j,m}\right)^T$, then the distance between x_i and x_j is defined as:

- Correlation Coefficient:

$$d\,(x_i, x_j) = \frac{\sum_{k=1}^{m}(x_{i,k} - \overline{x_i})(x_{j,k} - \overline{x_j})}{\left[\sum_{k=1}^{m}\left(x_{i,k} - \overline{x_i}\right)^2 \sum_{k=1}^{m}\left(x_{j,k} - \overline{x_j}\right)^2\right]^{1/2}}$$

$$\text{And } \overline{x_i} = \frac{1}{m}\sum_{k=1}^{m} x_{i,k}\,, \overline{x_j} = \frac{1}{m}\sum_{k=1}^{m} x_{j,k}$$

Higher value, higher similarity

Clustering: Basics

- Distance (dissimilarity) measures: given sample set $X = \{x_1, x_2, \dots, x_n\}, x_i \in \mathcal{R}^m$. $x_i, x_j \in X, x_i = \left(x_{i,1}, x_{i,2}, \dots x_{i,m}\right)^T, x_j = \left(x_{j,1}, x_{j,2}, \dots, x_{j,m}\right)^T$, then the distance between x_i and x_j is defined as:

- Cosine Distance: (1 - Cosine Similarity)

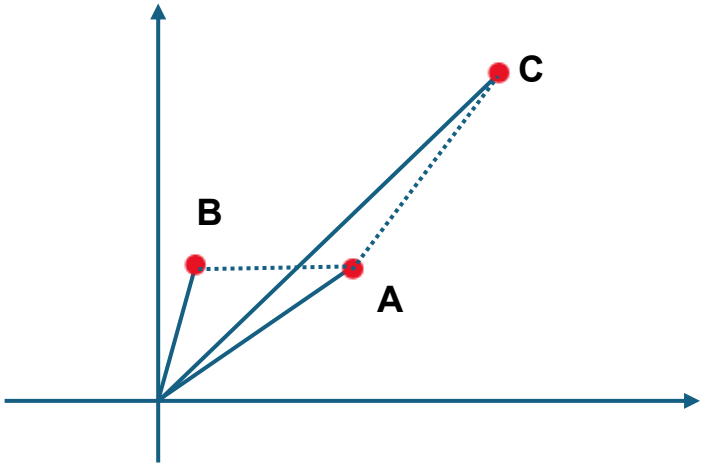
$$d\left(x_i, x_j\right) = 1 - \frac{\sum_{k=1}^{m} x_{i,k} x_{j,k}}{\left[\sum_{k=1}^{m}\left(x_{i,k}\right)^2 \sum_{k=1}^{m}\left(x_{j,k}\right)^2\right]^{1/2}}$$

Higher value, higher similarity

Clustering: Basics

- Distance (dissimilarity) measures: given sample set $X = \{x_1, x_2, \dots, x_n\}, x_i \in \mathcal{R}^m$. $x_i, x_j \in X, x_i = \left(x_{i,1}, x_{i,2}, \dots x_{i,m}\right)^T, x_j = \left(x_{j,1}, x_{j,2}, \dots, x_{j,m}\right)^T$, then the distance between x_i and x_j is defined as:

- We may have different results using different distance measures.



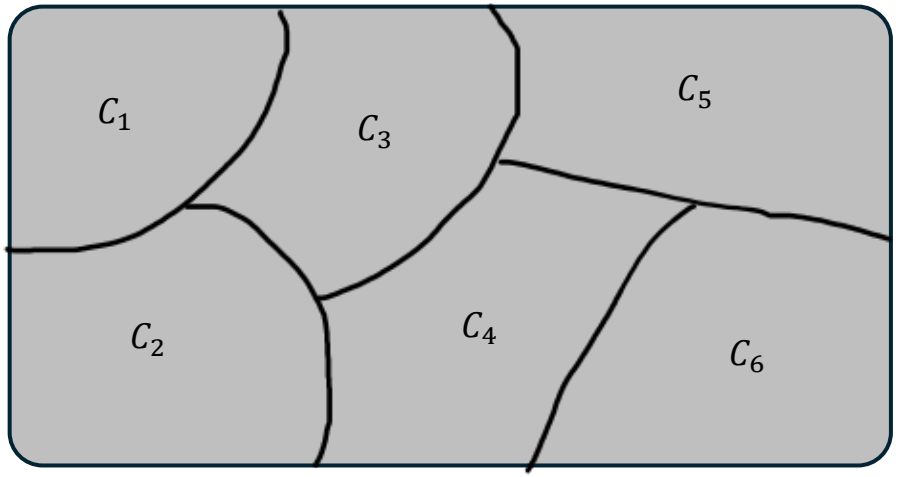
- **Euclidean distance:** $d(A, B) < d(A, C)$
- **Cosine distance:** $d(A, B) > d(A, C)$

Cluster: Basics

Given a dataset with n points in a d-dimensional space $D = \{x_i\}_{i=1}^{n}$, and given the number of desired clusters k, the goal of clustering is to partition the dataset into k groups or clusters, which is called a clustering and is denoted as $C = \{C_1, C_2, \dots, C_k\}$.

In each clustering C_i, there exists a positive number T that for any $x_i, x_j \in C_i$,

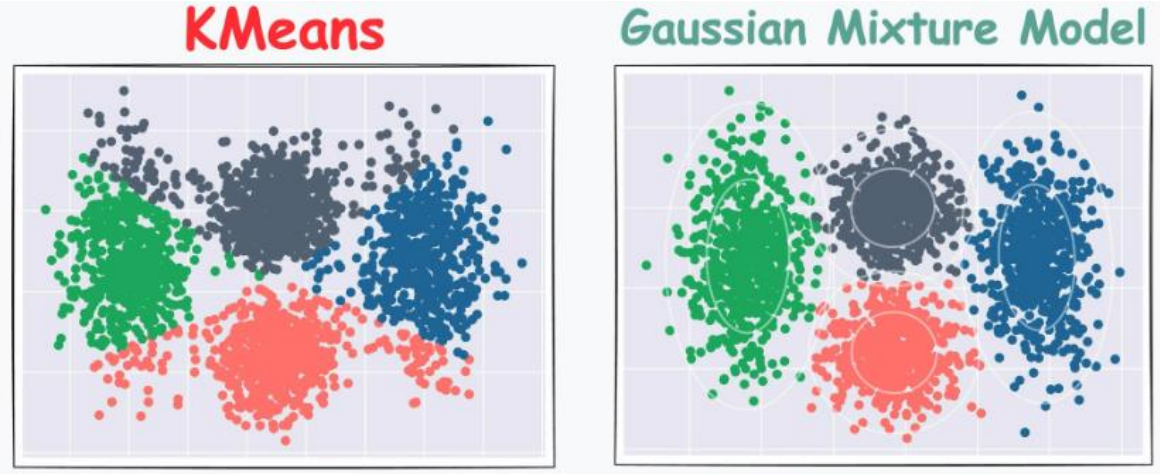
$$d(x_i, x_j) \leq T$$



Clustering: Basics

Hard Clustering vs. Soft Clustering:

- Hard Clustering: one data point is allocated to one and only one cluster, or the intersection of clusters are empty $C_i \cap C_j = \emptyset$; (K-means)
- Soft Clustering: one data point can be allocated to more than one clusters, or the intersection of clusters are not empty. (GMM)

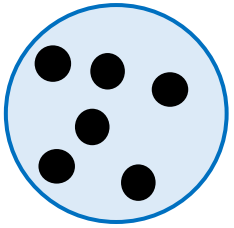


Clustering: Basics

Clustering properties:

- Clustering Mean value $\bar{x}_{C_i}$:

$$\bar{x}_{C_i} = \frac{1}{|C_i|}\sum_{i=1}^{|C_i|} x_i$$



- Clustering Diameter D_{C_i}:

$$D_{C_i} = \max_{x_i, x_j \in C_i} d(x_i, x_j)$$

- Clustering scatter matrix A_{C_i} and covariance matrix S_{C_i}:

$$A_{C_i} = \sum_{i=1}^{|C_i|} (x_i - \bar{x}_{C_i})(x_i - \bar{x}_{C_i})^T$$

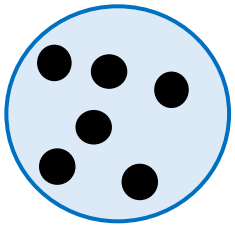
$$S_{C_i} = \frac{1}{d-1} A_{C_i}$$

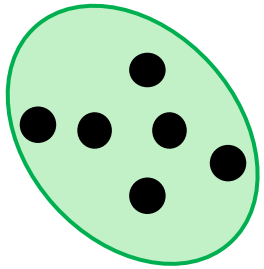
Where d is the dimension of features.

Cluster Distances

There are four different measures to compute the distance between each pair of clusters:

- Min distance (single linkage)
- Max distance (complete linkage)
- Average distance
- Mean distance

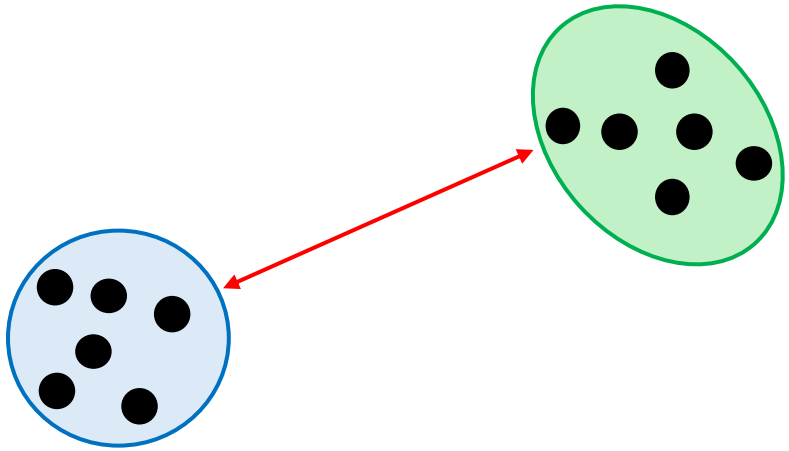




The Goal of Clustering

The core idea is simple:

- Find groups where …
  - High Intra-cluster Similarity: Points inside a cluster are very similar to each other.
  - Low Inter-cluster Similarity: Points in different clusters are very different from each other.



K-means Clustering

This is the most famous, popular, and widely-used clustering algorithm.

- **Type**: Partitioning algorithm.
- **The Goal**: To partition all your data points into K pre-defined, non-overlapping groups.
- **The Catch**: You *must* tell the algorithm how many clusters (K) you want to find *before* you start.

The Big Idea:

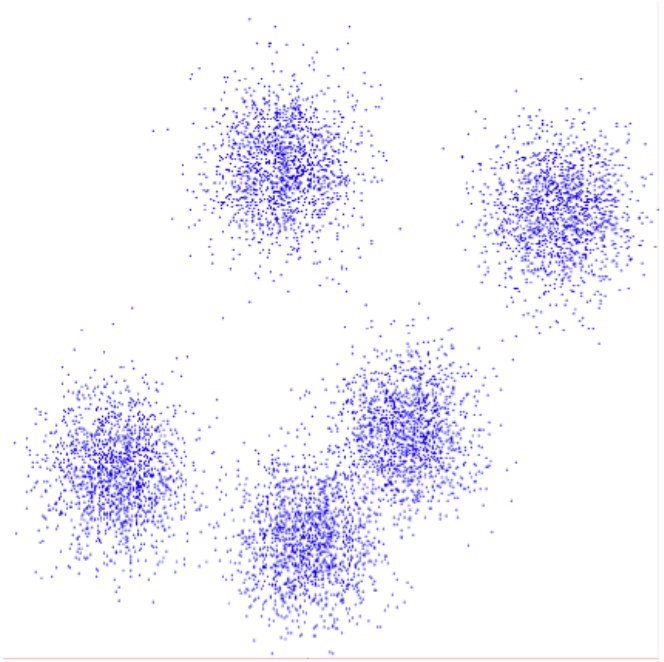
I. Each cluster is defined by its *center*, called the centroid.

II. Each data point is assigned to the cluster whose centroid it is *closest* to.

K-means

An iterative clustering algorithm

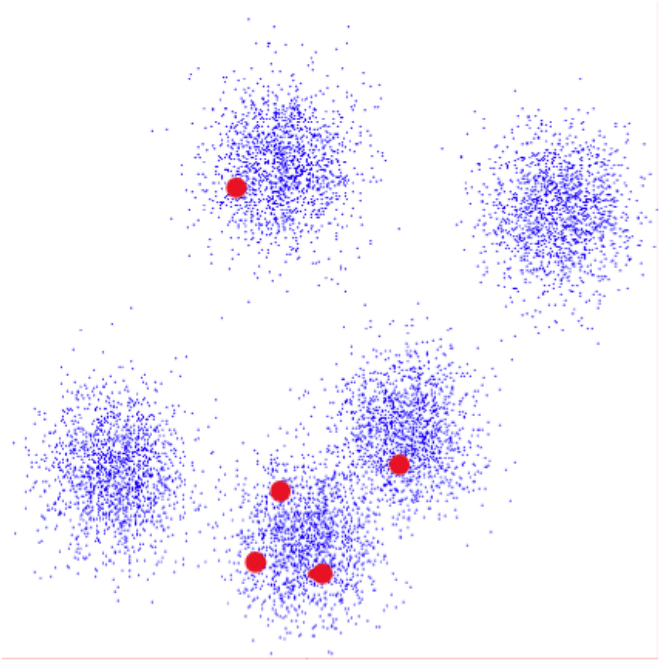
- Initialize: pick K random points as cluster centers



K-means

An iterative clustering algorithm

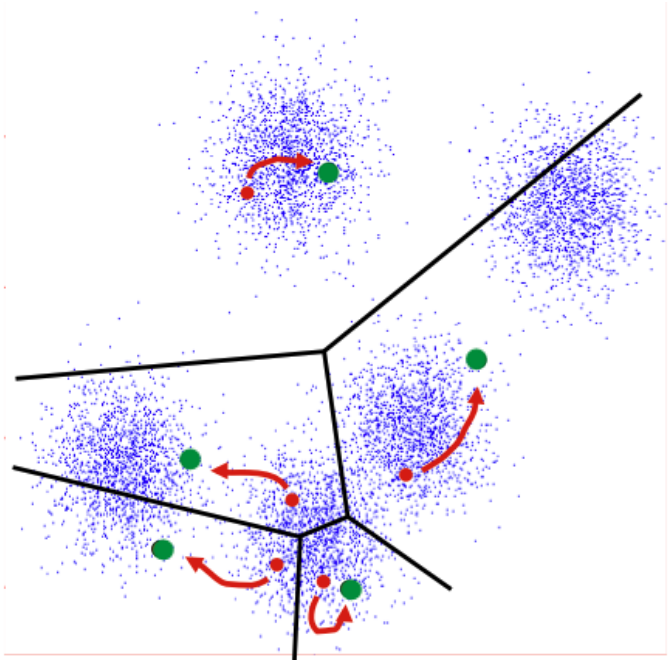
- Initialize: pick K random points as cluster centers
- Alternate:
  1. Assign data points to closest cluster center
  2. Change the cluster center to the average of its assigned points.



K-means

An iterative clustering algorithm

- Initialize: pick K random points as cluster centers
- Alternate:
  1. Assign data points to closest cluster center
  2. Change the cluster center to the average of its assigned points.
- Stop when no points' assignments change.



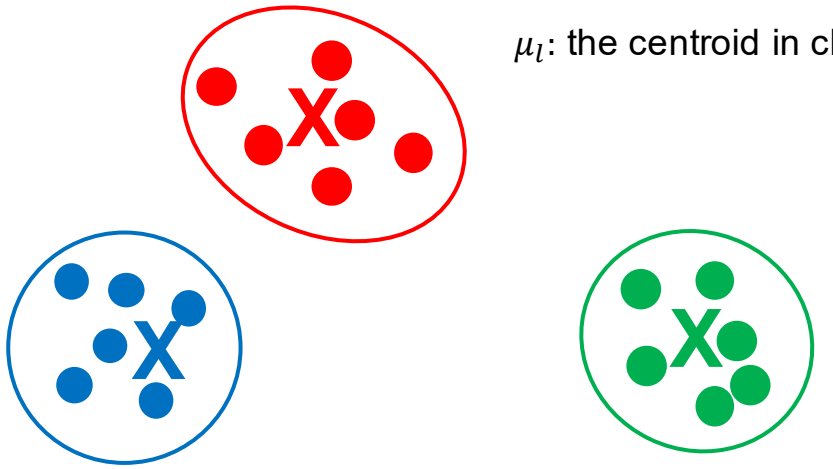
Properties of K-means algorithm

- Guaranteed to converge in a finite number of iterations.
- Running time per iteration:
  1. Assign data points to closest cluster center: $O(KN)$ time
  2. Change the cluster center to the average of its assigned points: $O(N)time$

K-means cost function

Loss function for a given partition $C_1, C_2, \ldots, C_k$:

$$L(C_1, C_2, \ldots, C_k) = \sum_{l=1}^{k} \sum_{i=1}^{|C_l|} ||x_i - \mu_l||^2$$



μ_l: the centroid in cluster C_l

K-means cost function

Objective:

$$arg\min_{C} \sum_{l=1}^{k} \sum_{i=1}^{|C_l|} ||x_i - \mu_l||^2$$

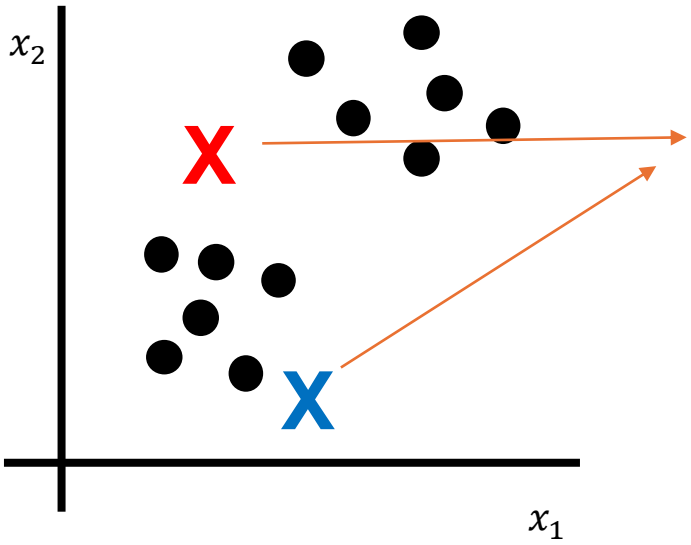
- How to we minimize $L(C)$?
  - If centroids known, can assign cluster.
  - If cluster known, can recompute centroids.

  - We use an iterative procedure.

K-means: minimizing the loss function

- E-step: Fix μ_l (centroid), minimize $J(.)$ w.r.t C (clustering):
  - Assign each data point to its nearest centroid
- M-step: Fix C (clustering), minimize $J(.)$ w.r.t μ_l (centroid)
  - Recompute centroids: set each centroid to the mean of the points in that cluster.
- The procedure is guaranteed to converge
  - K-means takes an alternating optimization approach, each step is guaranteed to decrease the objective – thus guaranteed to converge.
- Converges to a local minimum:
  - Use different initializations and pick the best solution
  - May still be insufficient for large search spaces

The K-means algorithm

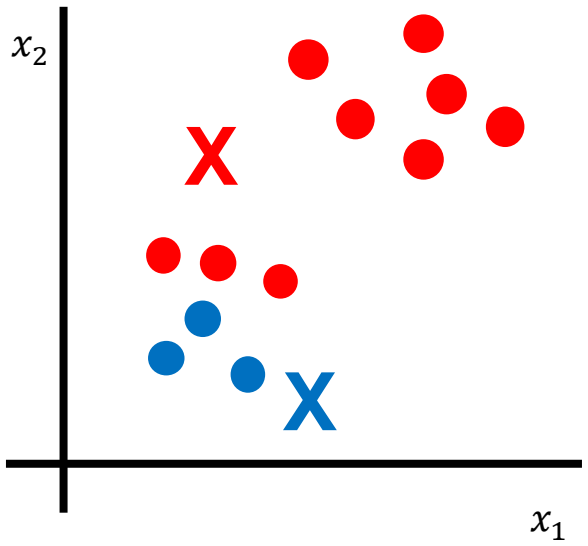


STEP 1:
Initialize centroids: pick K random points as cluster centers (means)

a. Could be any point in the plane
b. Typically, you pick two observations in your training data

Shown here for K=2

The K-means algorithm

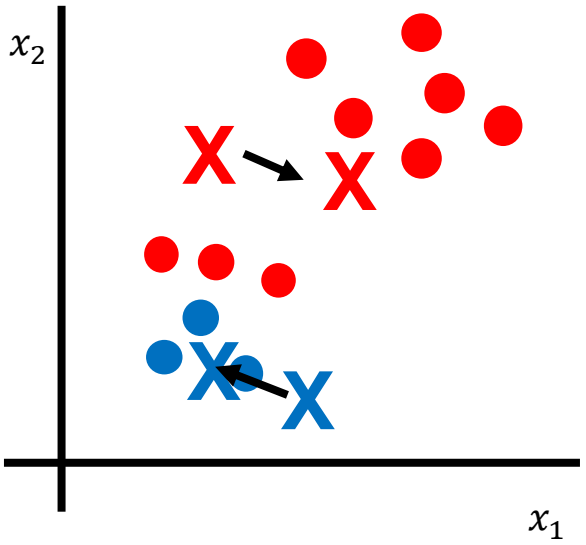


STEP 2:
Find closest centroids

a. Calculate the distance of each point to the centroids
b. 'Assign' each point to the centroid it is closest to.

The points assigned to a given centroid define the 'cluster'

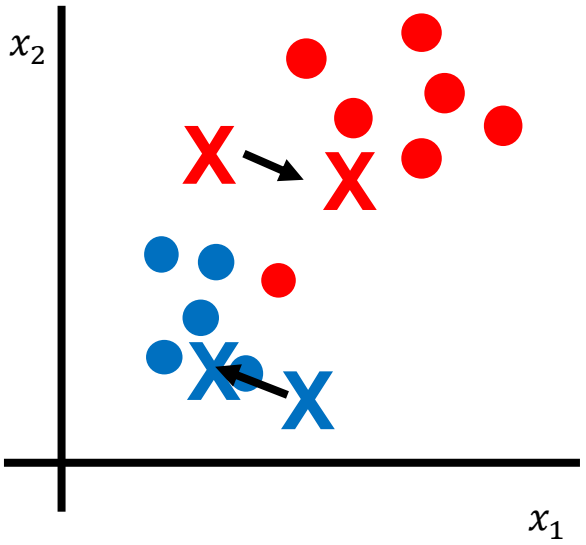
The K-means algorithm



STEP 3:
Recompute cluster centroids

Calculate the average of all the points within each cluster.
This defines the new centroid locations

The K-means algorithm

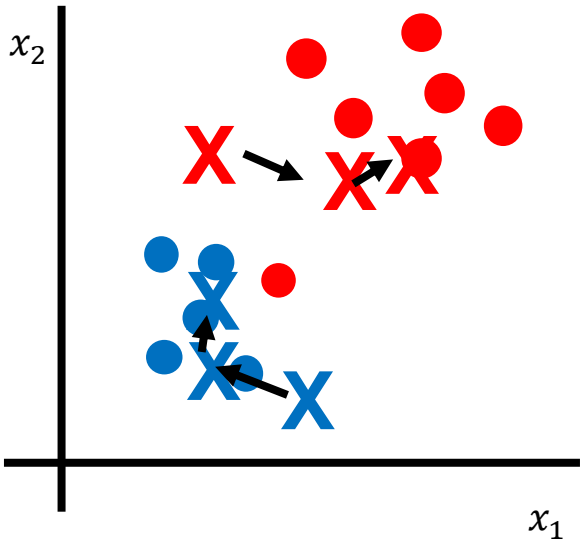


STEP 4:
Iteratively do steps 2 and 3

Find closest centroids

Recompute centroids

The K-means algorithm

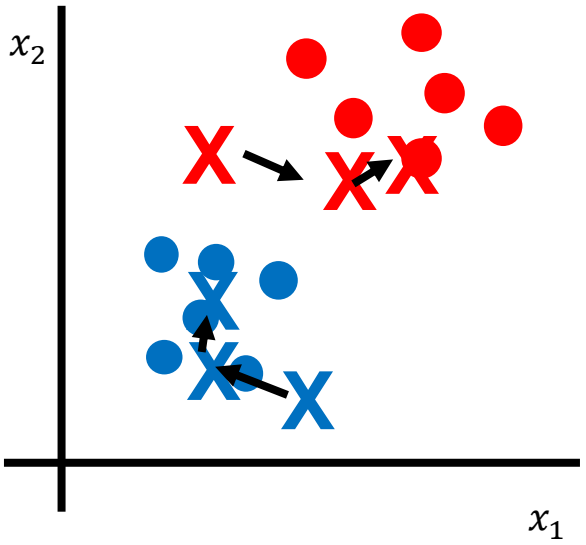


STEP 4:
Iteratively do steps 2 and 3

Find closest centroids

Recompute centroids

The K-means algorithm

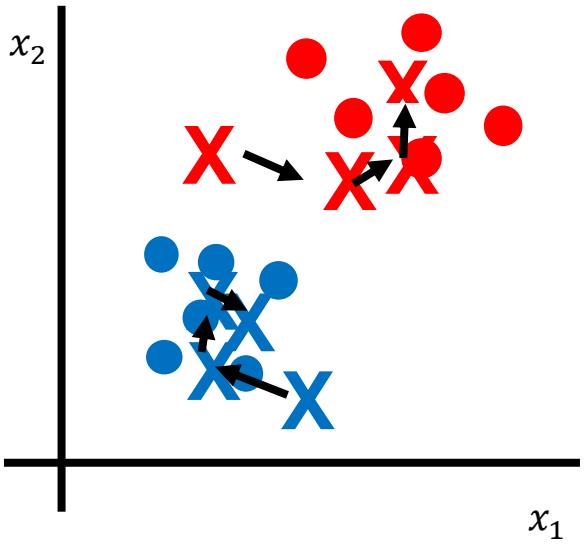


STEP 4:
Iteratively do steps 2 and 3

Find closest centroids

Recompute centroids

The K-means algorithm

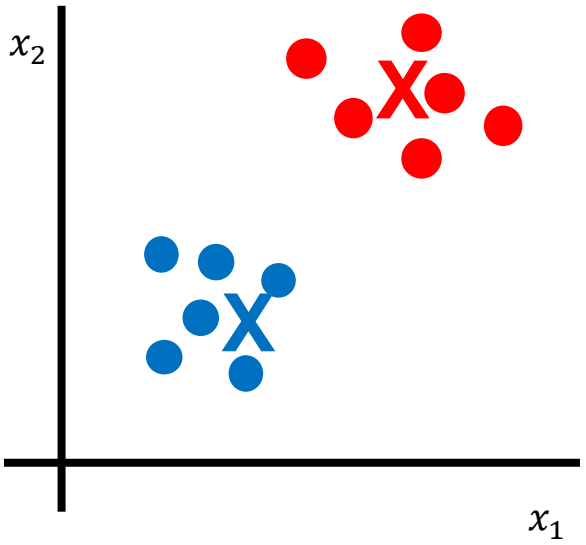


STEP 4:
Iteratively do steps 2 and 3

Find closest centroids

Recompute centroids

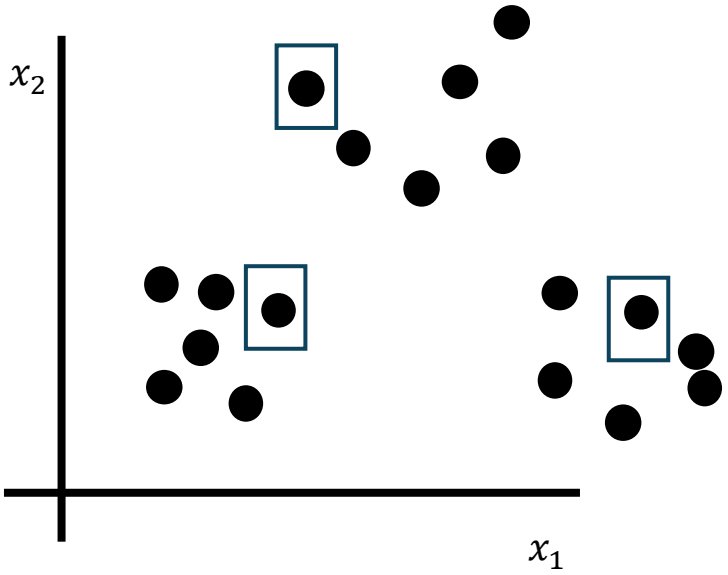
The K-means algorithm



STEP 5:
Iteratively do steps 2 and 3

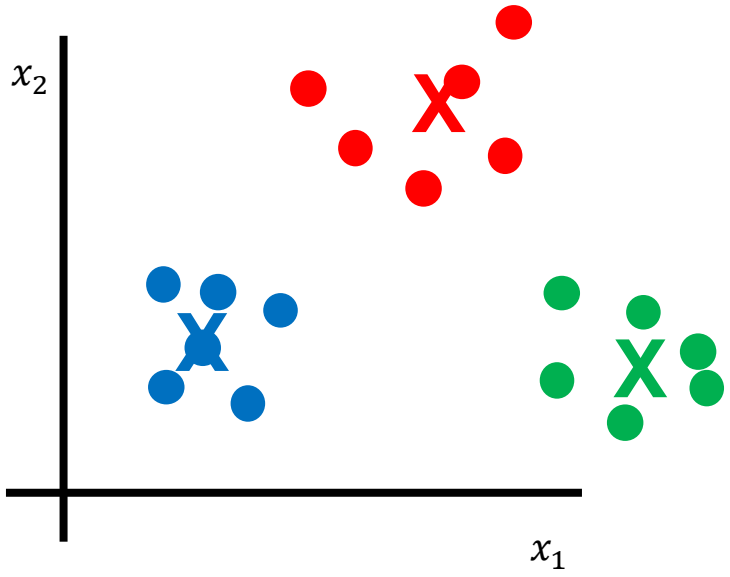
STOP when no change in cluster assignment

Random initialization May lead to Different Clusters (1)

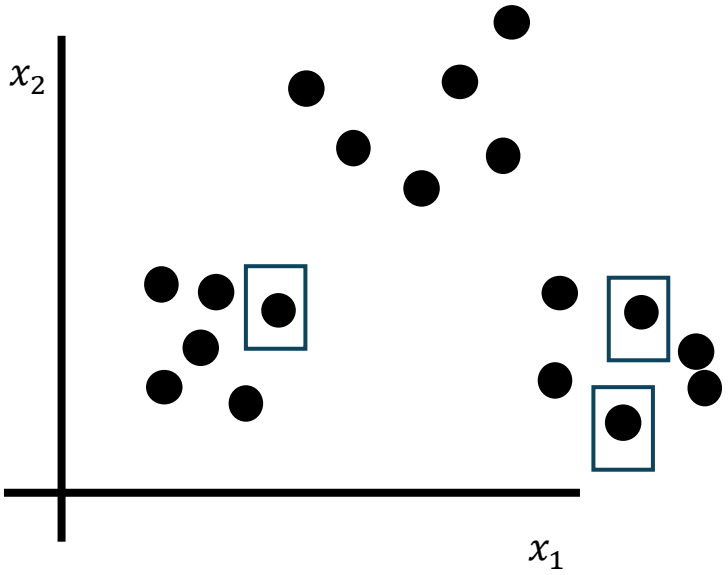


Randomly pick K observations in your dataset as your initial centroids

Random initialization May lead to Different Clusters (2)



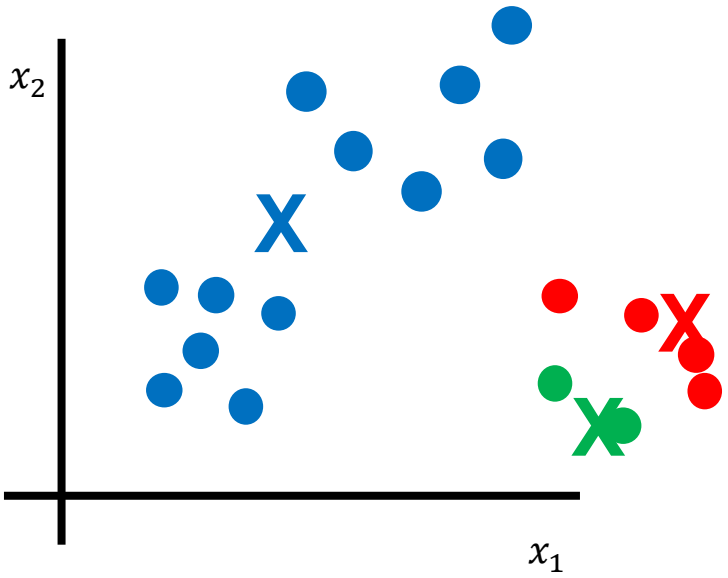
Random initialization May lead to Different Clusters (3)



But for certain choices of initial centroids, you can get stuck in a local optima

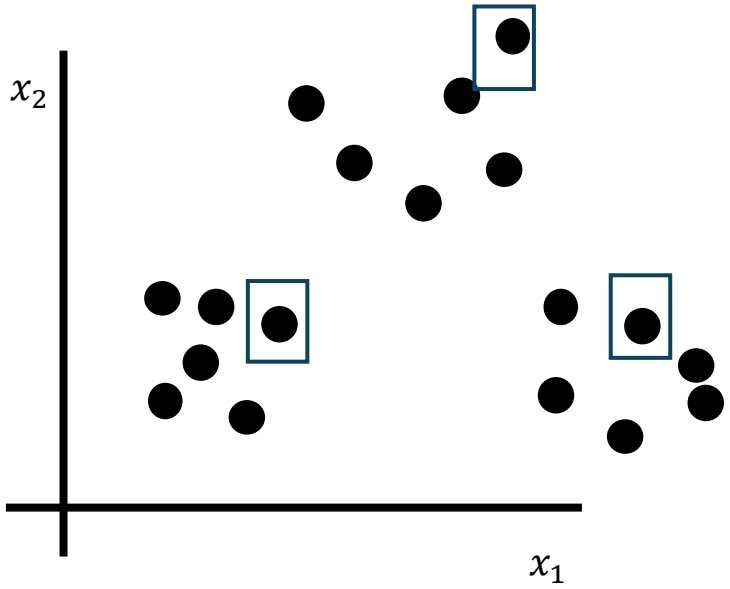
Random initialization May lead to Different Clusters (4)

Issue: Local optima



But for certain choices of initial centroids, you can get stuck in a local optima

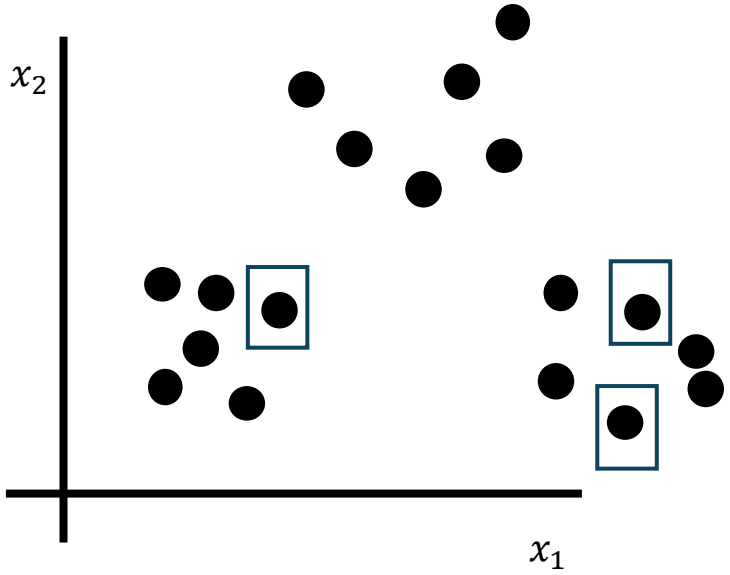
Solution: Multiple random initializations



Run K-means several times with different initial centroids. Compute $J(\ldots)$ each time

Trial number	Minimized $J(\ldots)$ value
1	23.5

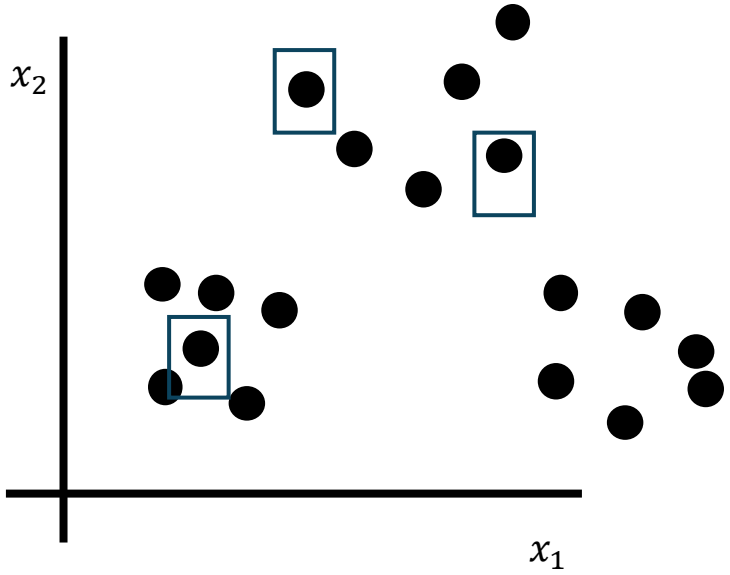
Solution: Multiple random initializations



Run K-means several times with different initial centroids. Compute $J(\ldots)$ each time

Trial number	Minimized $J(\ldots)$ value
1	23.5
2	176.2

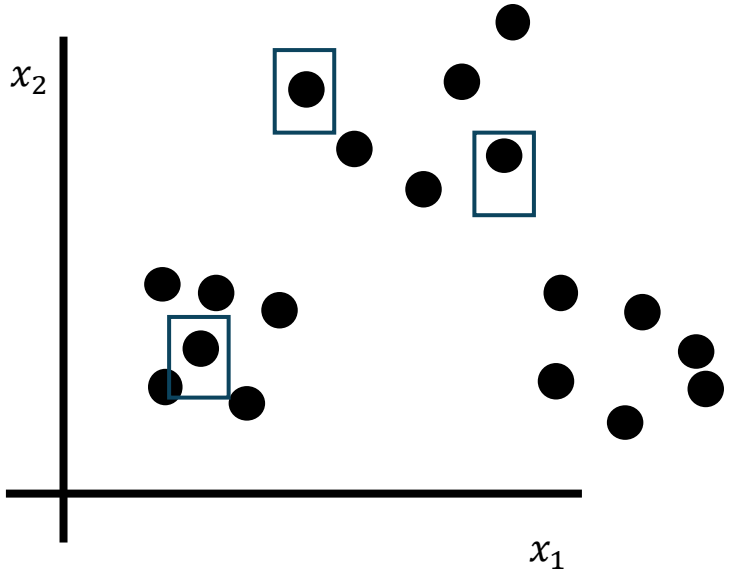
Solution: Multiple random initializations



Run K-means several times with different initial centroids. Compute $J(\ldots)$ each time

Trial number	Minimized $J(\ldots)$ value
1	23.5
2	176.2
…	
50	39.1

Solution: Multiple random initializations



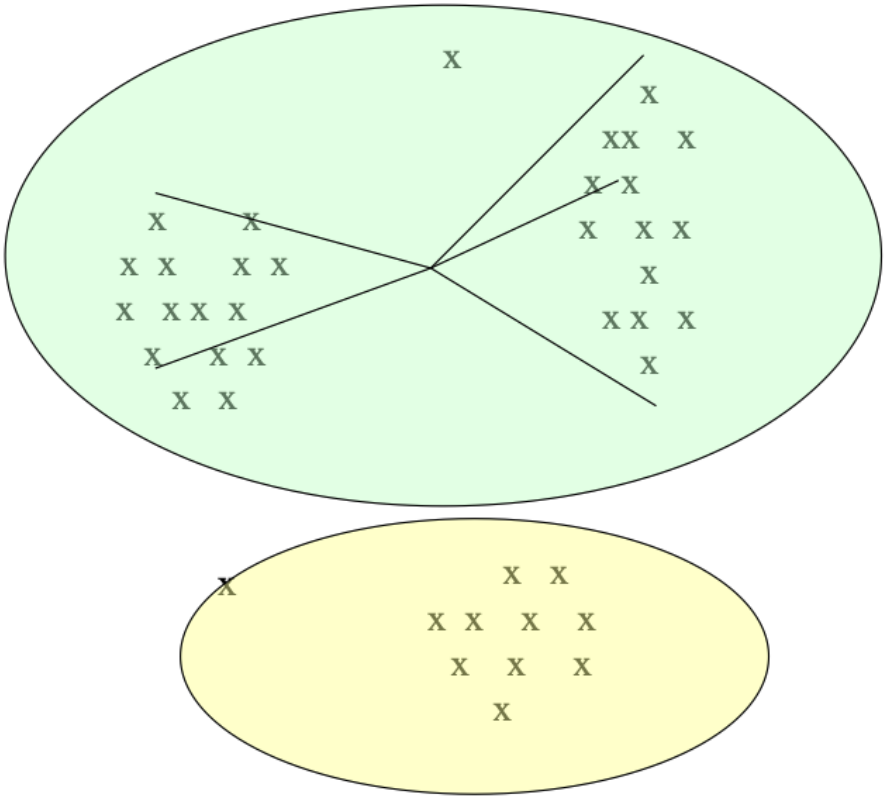
Run K-means several times with different initial centroids. Compute $J(\ldots)$ each time

Trial number	Minimized $J(\ldots)$ value
1	23.5
2	176.2
…	
50	39.1

Pick the clustering assignment which gave the lowest $J(\ldots)$ value

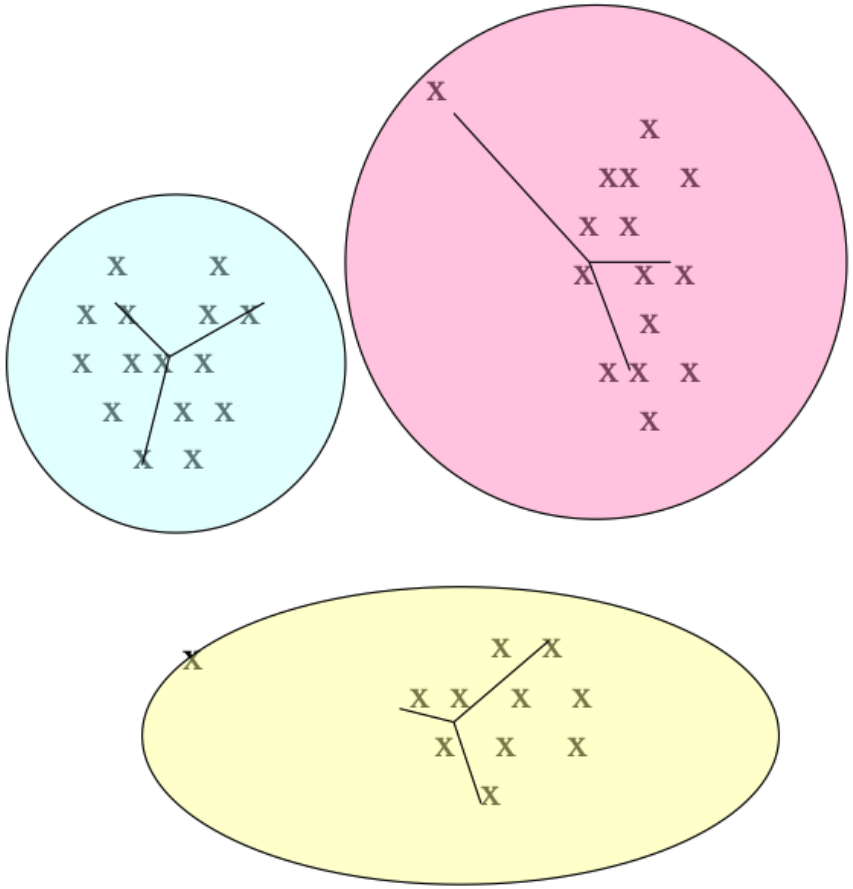
Example: picking K

Too few;
many long
distances
to centroid



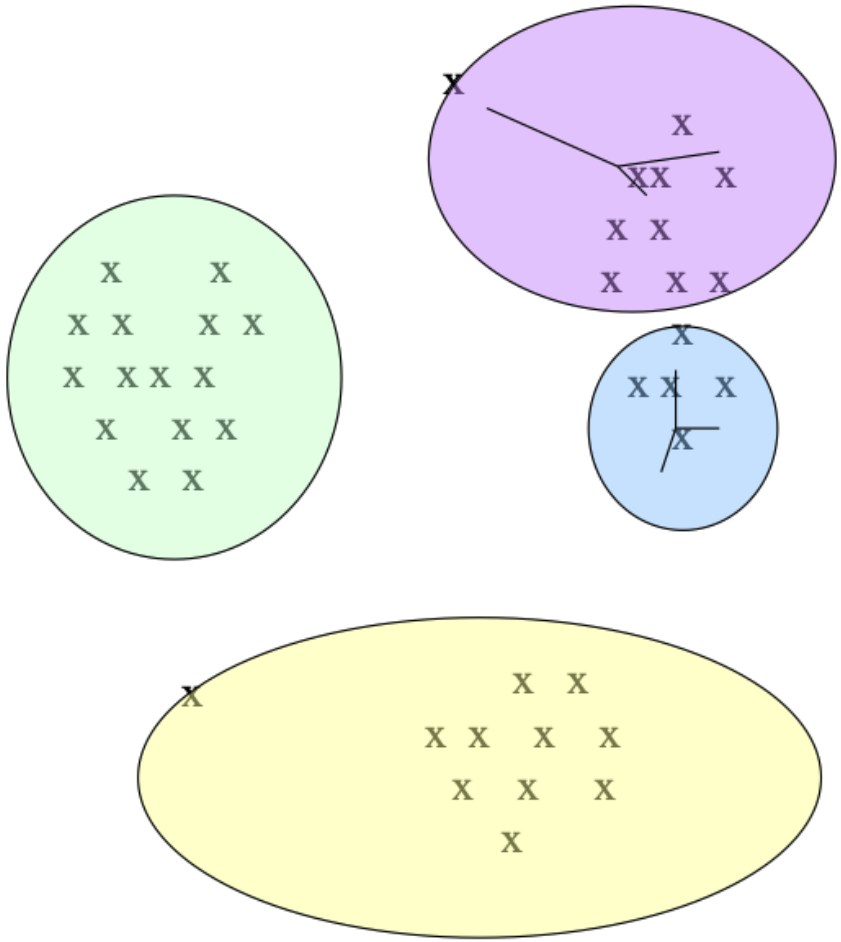
Example: picking K

Just right;
distances
rather short



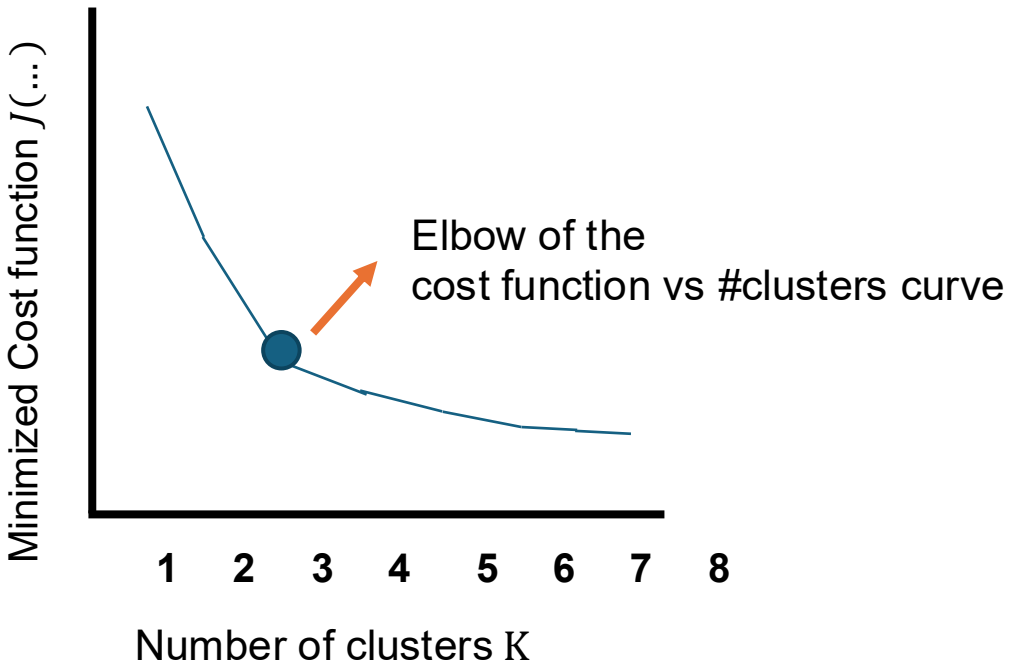
Example: picking K

Too many;
little improvement
in average
distance



How to choose K: Elbow method

- Try different K, looking at the change in the average distance to centroid as K increases
- Average falls rapidly until right K, then changes little.



Question

Suppose you run k-means using k = 3 and k = 5. You find that the cost function J is much higher for k = 5 than for k = 3. What can you conclude?

A) This is mathematically impossible. There must be a bug in the code.

B) The correct number of clusters is k = 3.

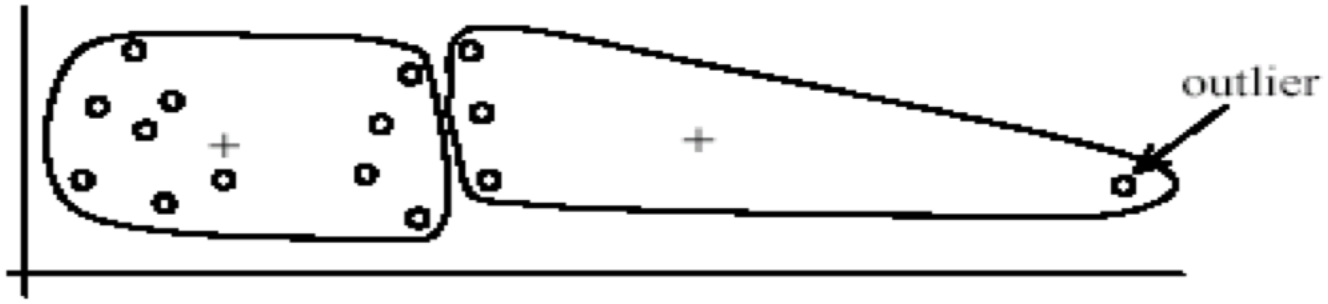
C) In the run with k = 5, k-means got stuck in a bad local minimum. You should try re-running k-means with multiple random initializations.

D) In the run with k = 3, k-means got lucky. You should try re-running k-means with k = 3 and different random initializations until it performs no better than with k = 5.

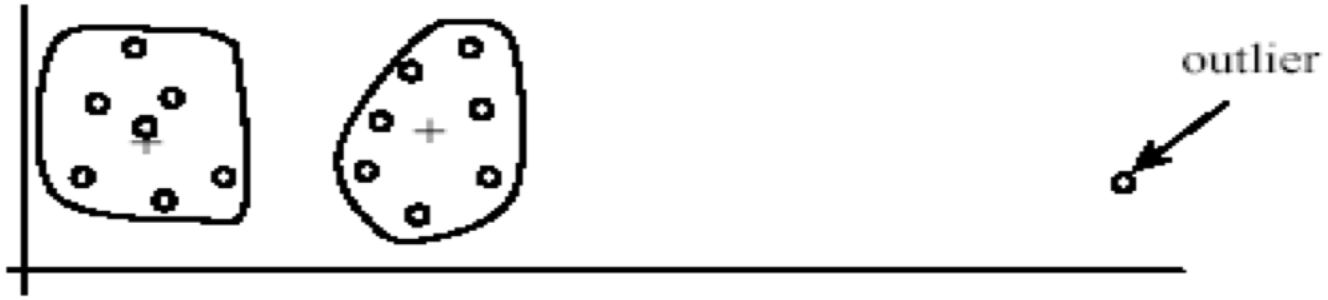
Weakness of K-means

- The user needs to specify k.
- Hard assignments of data points to clusters can cause a small perturbation to a data point to flip it to another cluster.
- Assumes spherical clusters and equal probabilities for each cluster.
- The algorithm is sensitive to outliers
  - Outliers are data points that are very far away from other data points.
  - Outliers could be errors in the data recording or some special data points with very different values.
- Works poorly on non-convex clusters.

Outliers

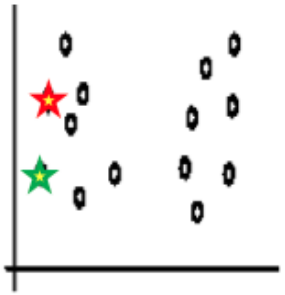


(A): Undesirable clusters

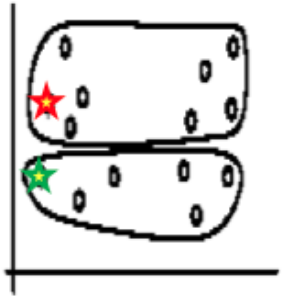


(B): Ideal clusters

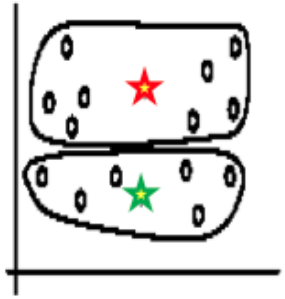
Sensitivity to initial seeds



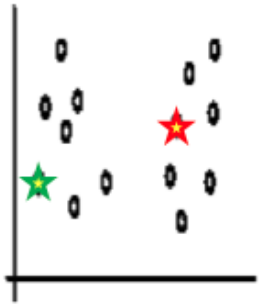
Random selection of seeds (centroids)



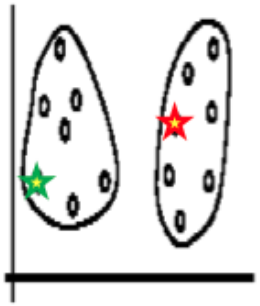
Iteration 1



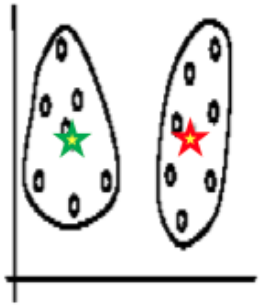
Iteration 2



Random selection of seeds (centroids)



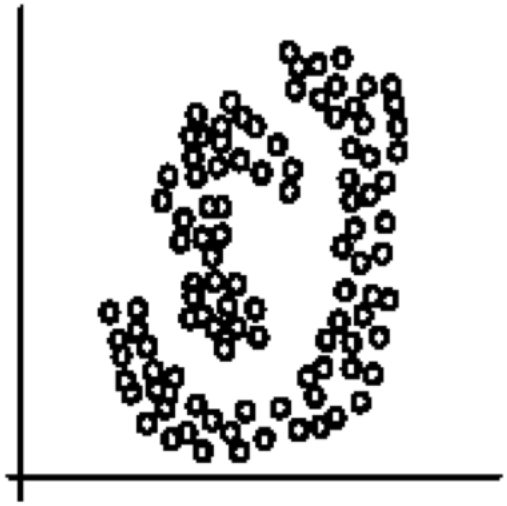
Iteration 1



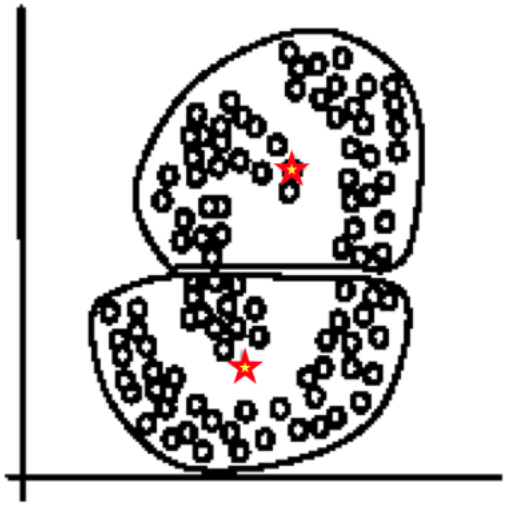
Iteration 2

Special data structures

- The k-means algorithm is not suitable for discovering clusters that are not spherical.



(A): Two natural clusters



(B): *k*-means clusters

Summary of K-Means

- We've moved from Supervised (Prediction) to Unsupervised (Discovery)
- Clustering finds natural, hidden groups in our data.
- Hierarchical Clustering builds a "tree" (dendrogram) and is good for seeing all possible groupings, but it's slow.
- K-Means is the fast, popular algorithm due to its simplicity and efficiency
  - Algorithm: Initialize → Assign → Update → Repeat.
  - Challenge: You must choose K.
  - Solution: The Elow Method (plotting K vs. cost) helps you find a good K.

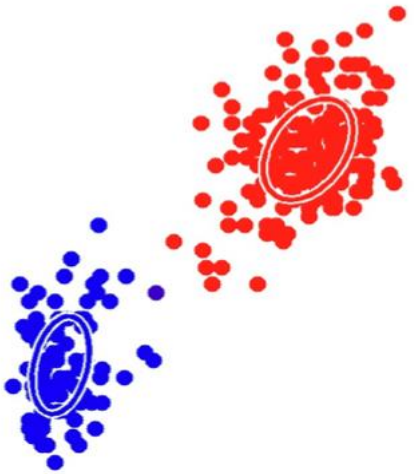
Soft Clustering

- Clustering typically assumes that each instance is given a “hard” assignment to exactly one cluster.
- Does not allow uncertainty in class membership or for an instance to belong to more than one cluster.
- *Soft clustering* gives probabilities that an instance belongs to each of a set of clusters.
- Each instance is assigned a probability distribution across a set of discovered categories (probabilities of all categories must sum to 1).

Mixtures of Gaussians

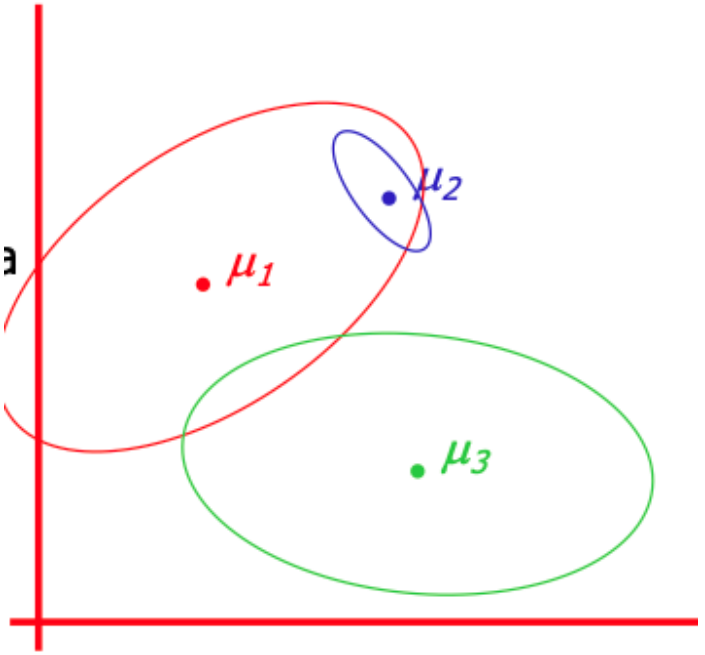
- K-means algorithm
  - Assigned each example to exactly one cluster
  - What if clusters are overlapping?
    - Hard to tell which cluster is right, and maybe we should try to remain uncertain.
  - What if cluster has non-circular shape?

- Gaussian mixture models:
  - Clusters modeled as Gaussians
    - Not just by their mean, but also variance
  - EM algorithm: assign data to cluster with some probability
  - Given probability model of x ("generative model")



Gaussian Mixture Model

- Gaussian Mixture Models (GMMs) assume that there are K Gaussian distributions, and each of these distributions represent a cluster (i.e., C_1, C_2, C_3).
- Each Gaussian with mean μ_i and covariance matrix $\sum_i$



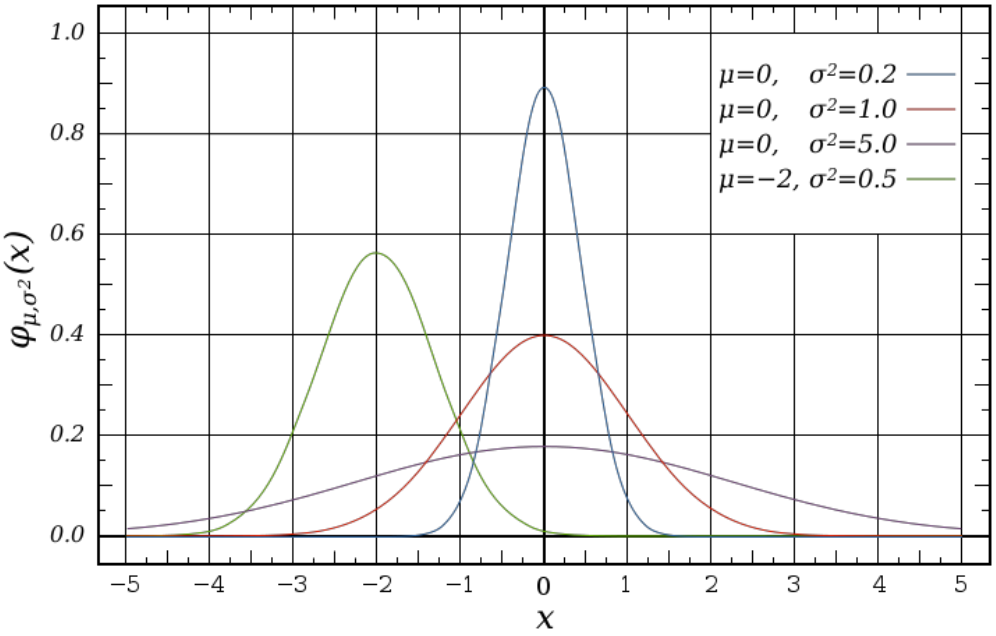
Assume that each datapoint is generated according to the following recipe:

1. Pick a cluster at random with probability $P(C_i)$;
2. Generate datapoint $\sim N(\mu_i, \sum_i)$;

Example: 1-d Gaussian Distribution

$$p(x \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{ -\frac{1}{2\sigma^2}(x-\mu)^2 \right\}$$

- μ is the mean
- σ^2 is the variance



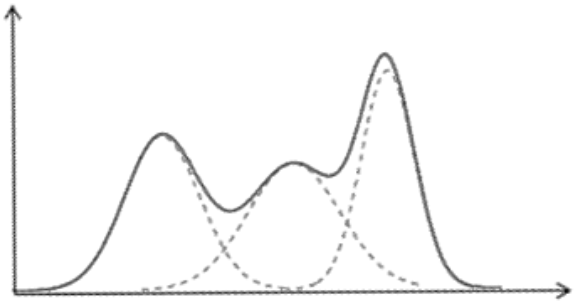
Mixture Models in 1-d

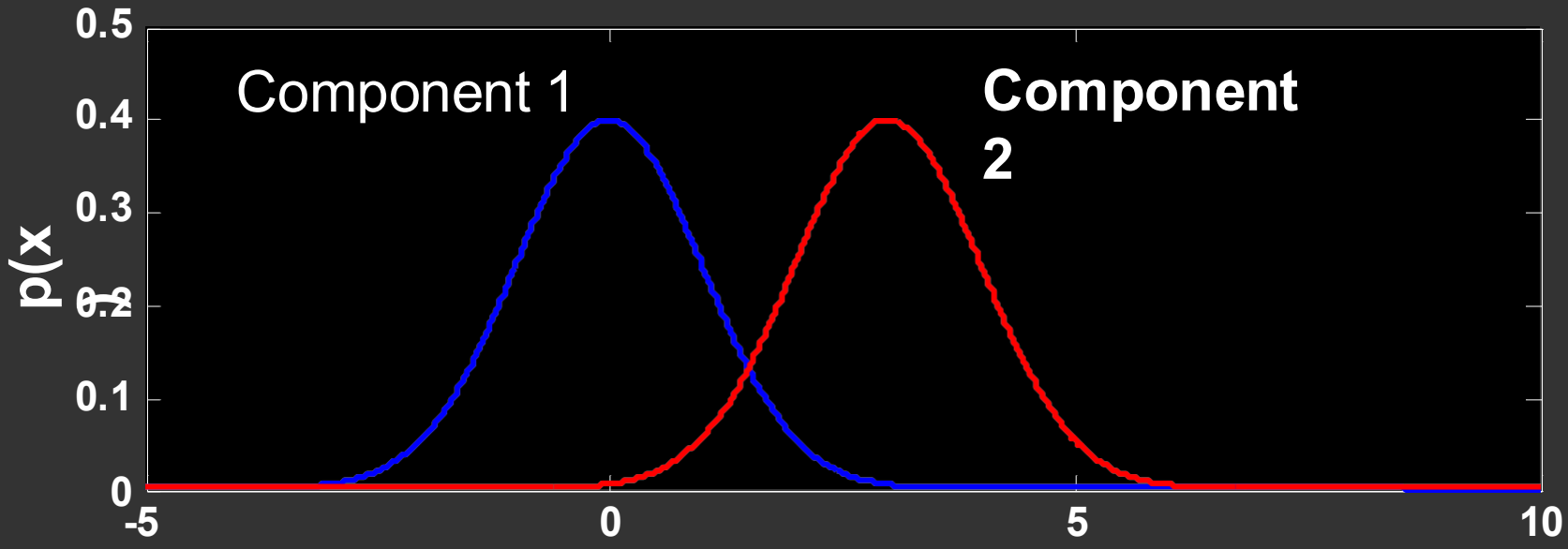
Formally a Mixture Model is the weighted sum of a number of pdfs where the weights are determined by a distribution, π

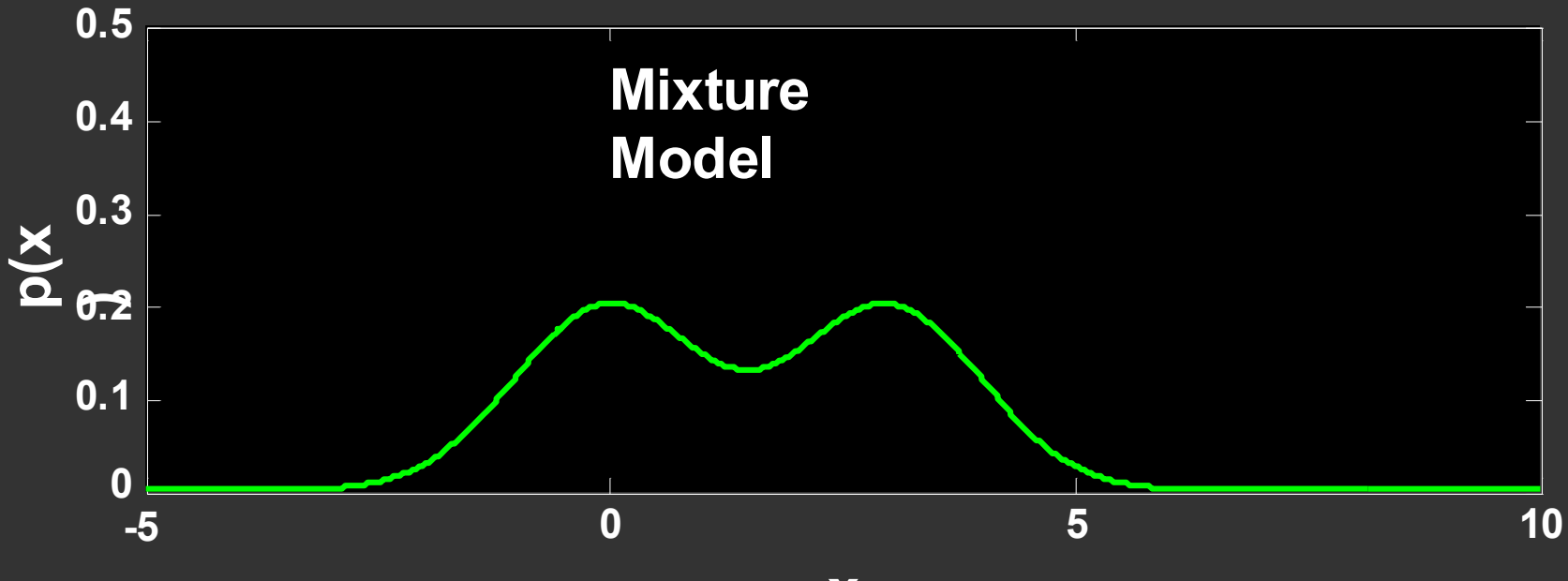
$$p(x) = \pi_0 f_0(x) + \pi_1 f_1(x) + \pi_2 f_2(x) + \ldots + \pi_k f_k(x)$$

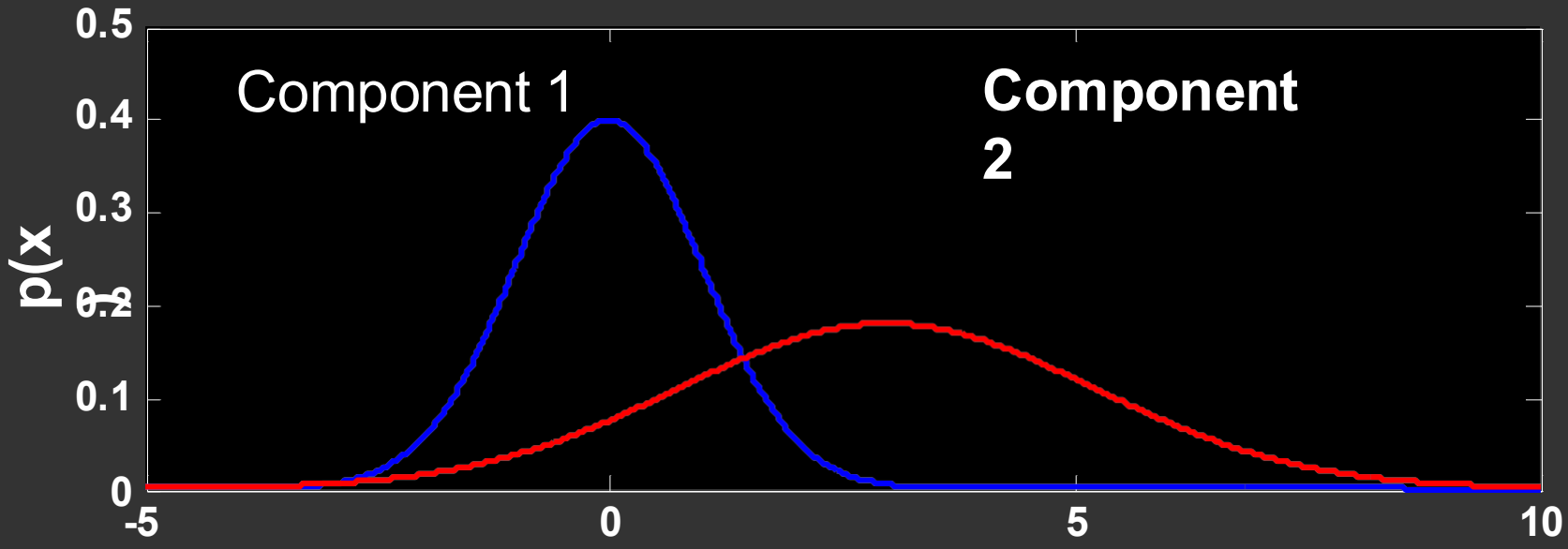
$$\text{where } \sum_{i=0}^{k} \pi_i = 1$$

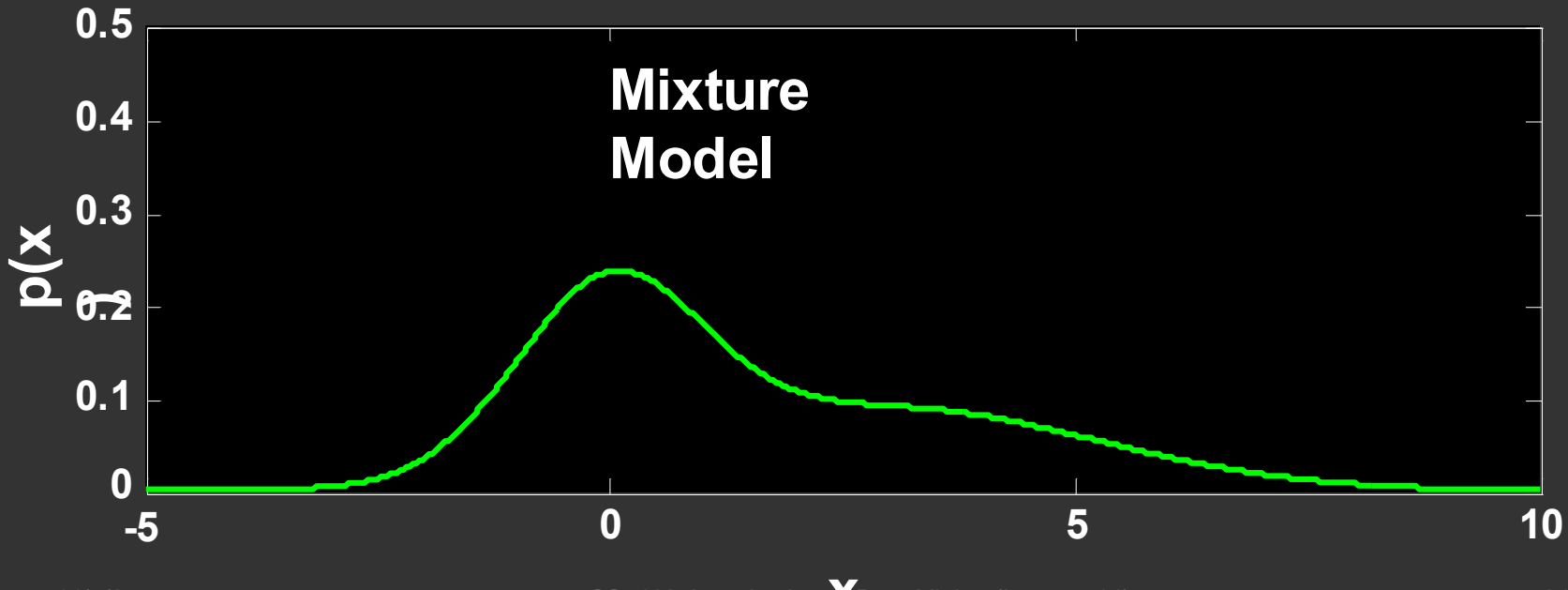
$$\boxed{p(x) = \sum_{i=0}^{k} \pi_i f_i(x)}$$

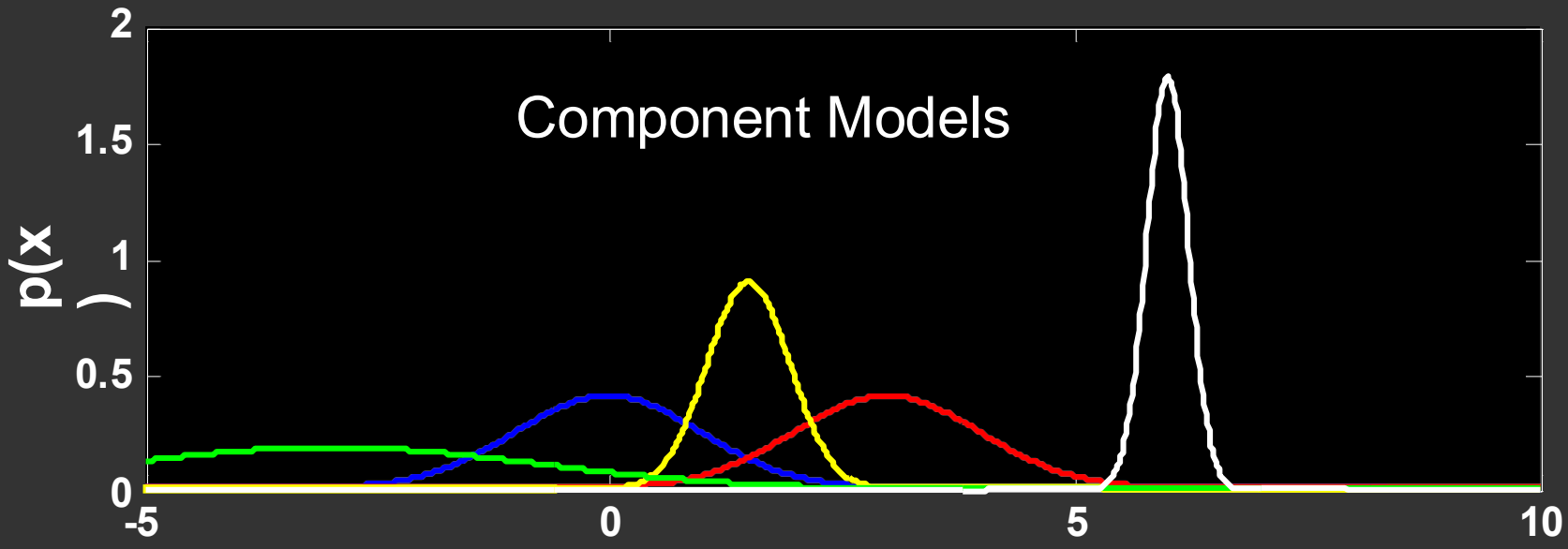


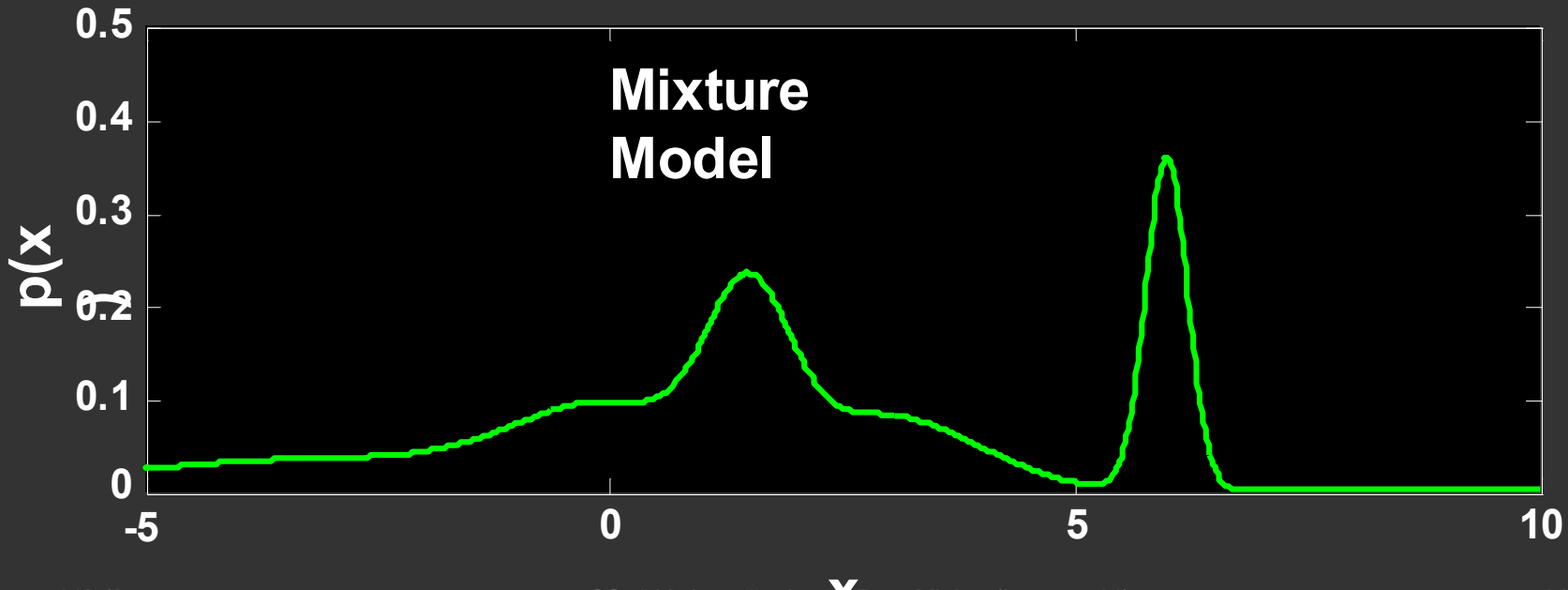

Component 1
Component 2
0.5
0.4
0.3
0.2
0.1
0
-5
0
5
10


Mixture Model
0.5
0.4
0.3
0.2
0.1
0
-5
0
5
10
x


Component 1
Component 2
0.5
0.4
0.3
0.2
0.1
0
-5
0
5
10


Mixture Model
0.5
0.4
0.3
0.2
0.1
0
-5
0
5
10
x


Component Models
p(x)
2
1.5
1
0.5
0
-5
0
5
10


Mixture Model
p(x)
0.5
0.4
0.3
0.2
0.1
0
-5
0
5
10
x

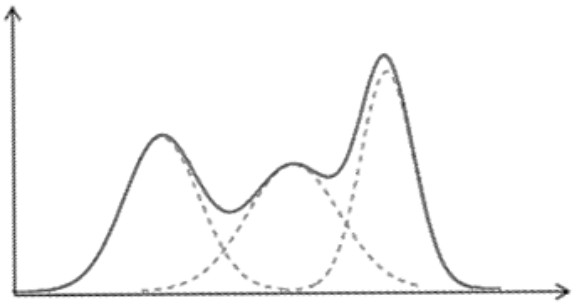
Gaussian Mixture Models

GMM: the weighted sum of a number of Gaussians where the weights are determined by a distribution, π

$$p(x) = \pi_0 N(x|\mu_0, \Sigma_0) + \pi_1 N(x|\mu_1, \Sigma_1) + \ldots + \pi_k N(x|\mu_k, \Sigma_k)$$

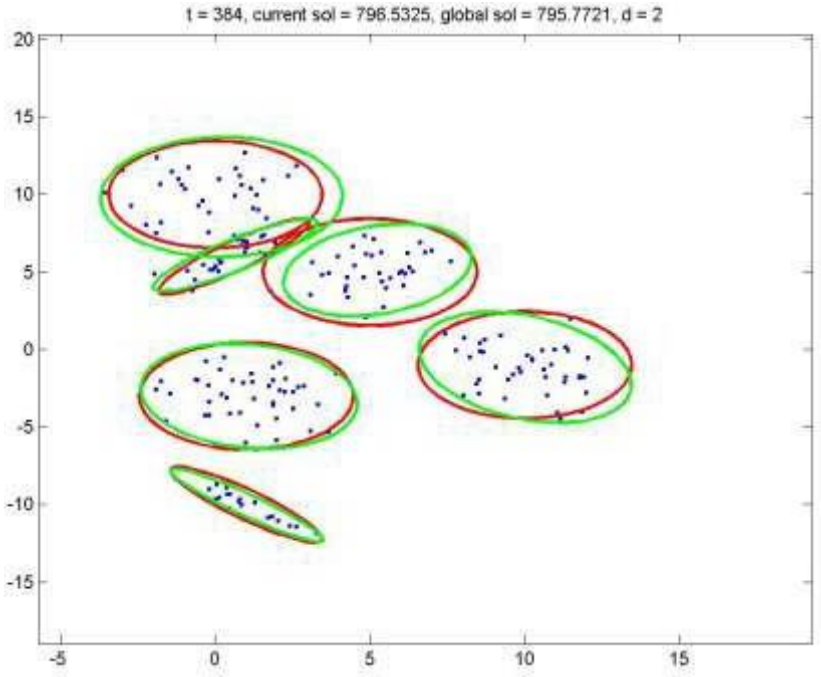
$$\text{where } \sum_{i=0}^{k} \pi_i = 1$$

$$p(x) = \sum_{i=0}^{k} \pi_i N(x|\mu_k, \Sigma_k)$$



Gaussian Mixture Models for Clustering

- Rather than identifying clusters by “nearest” centroids
- Fit a Set of *k* Gaussians to the data
- Maximum Likelihood over a mixture model



Example: GMM in one dimension

Assume that the following 1-D data come from 2 gaussian models:

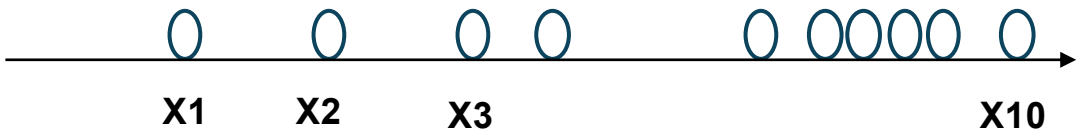
Which sources does each data come from?



Example: GMM in one dimension

Assume that the following 1-D data come from 2 gaussian models:

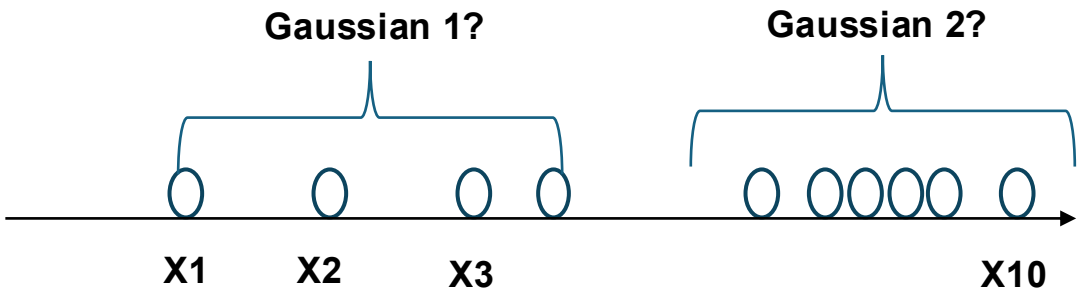
Which sources does each data come from?



Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data



Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data

- **Initialize parameters (randomly): start with two random gaussians:** $N(\mu_1, \sigma_1), N(\mu_2, \sigma_2)$ **and** $p(C_1), p(C_2)$**;**

Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data

- **Initialize parameters (randomly): start with two random gaussians:** $N(\mu_1, \sigma_1), N(\mu_2, \sigma_2)$ **and** $p(C_1), p(C_2)$**;**
- **E-step: assign probabilistic membership to all input samples** x_i**:**

$$p_{1i} = p(C_1|x_i) = \frac{p(x_i|C_1)\,p(C_1)}{p(x_i)} = \frac{p(x_i|C_1)\,p(C_1)}{p(x_i|C_1)\,p(C_1) + p(x_i|C_2)\,p(C_2)}$$

$$p_{2i} = p(C_2|x_i) = \frac{p(x_i|C_2)\,p(C_2)}{p(x_i)} = \frac{p(x_i|C_2)\,p(C_2)}{p(x_i|C_1)\,p(C_1) + p(x_i|C_2)\,p(C_2)}$$

Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data

- **Initialize parameters (randomly): start with two random gaussians:** $N(\mu_1, \sigma_1), N(\mu_2, \sigma_2)$ **and** $p(C_1), p(C_2)$**;**
- **M-step: re-estimate parameters based on probabilistic:**
  - **Mean:**

$$\mu_1 = \frac{\sum_j p_{1j} x_j}{\sum_j p_{1j}}; \qquad \mu_2 = \frac{\sum_j p_{2j} x_j}{\sum_j p_{2j}};$$

  - **Variance:**

$$\sigma_1 = \frac{\sum_j p_{1j} (x_j - \mu_1)^2}{\sum_j p_{1j}}; \quad \sigma_2 = \frac{\sum_j p_{2j} (x_j - \mu_1)^2}{\sum_j p_{2j}}$$

  - **Prior probability of cluster** C_i**:**

$$p(C_1) = \frac{\sum_j p_{1j}}{n}; \qquad p(C_2) = \frac{\sum_j p_{2j}}{n};$$

Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data

- **Initialize parameters (randomly): start with two random gaussians:** $N(\mu_1, \sigma_1), N(\mu_2, \sigma_2)$ **and** $p(C_1), p(C_2)$**;**

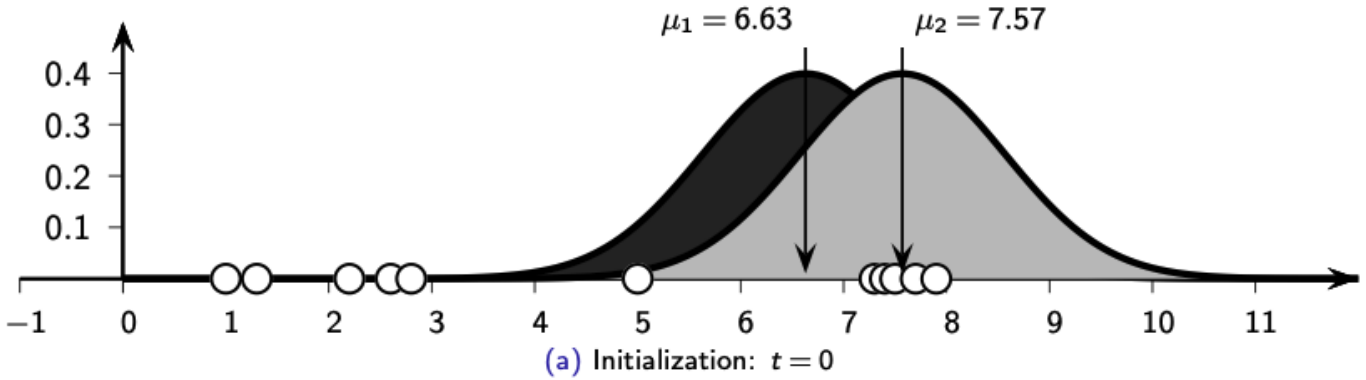
- **E-step: assign probabilistic membership to all input samples** x_i**:**
- **M-step: re-estimate parameters based on probabilistic:**

- **Repeat until change in parameters are smaller than a threshold.**

Example: GMM in one dimension

EM algorithm:

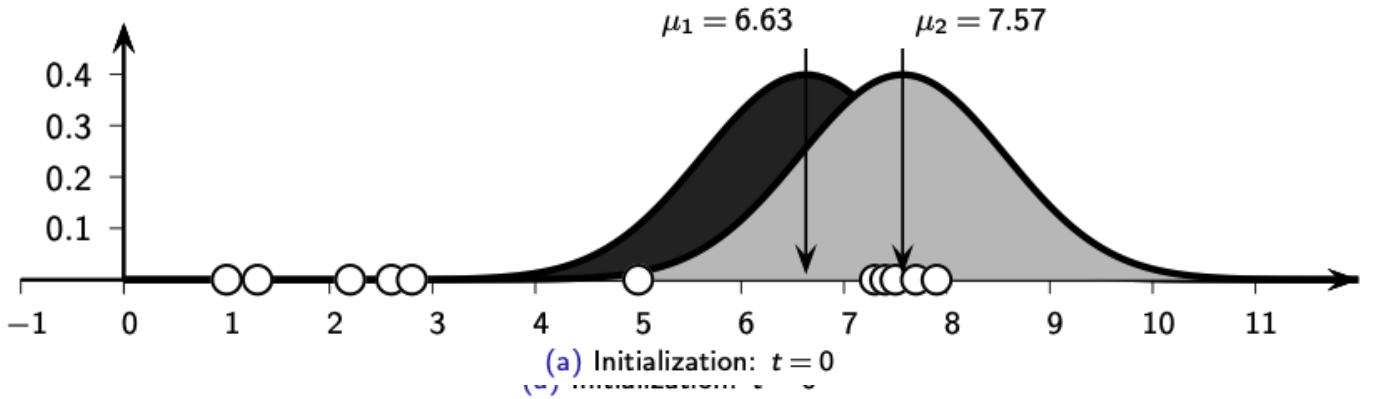
Find the best fit models to maximize the likelihood of the data



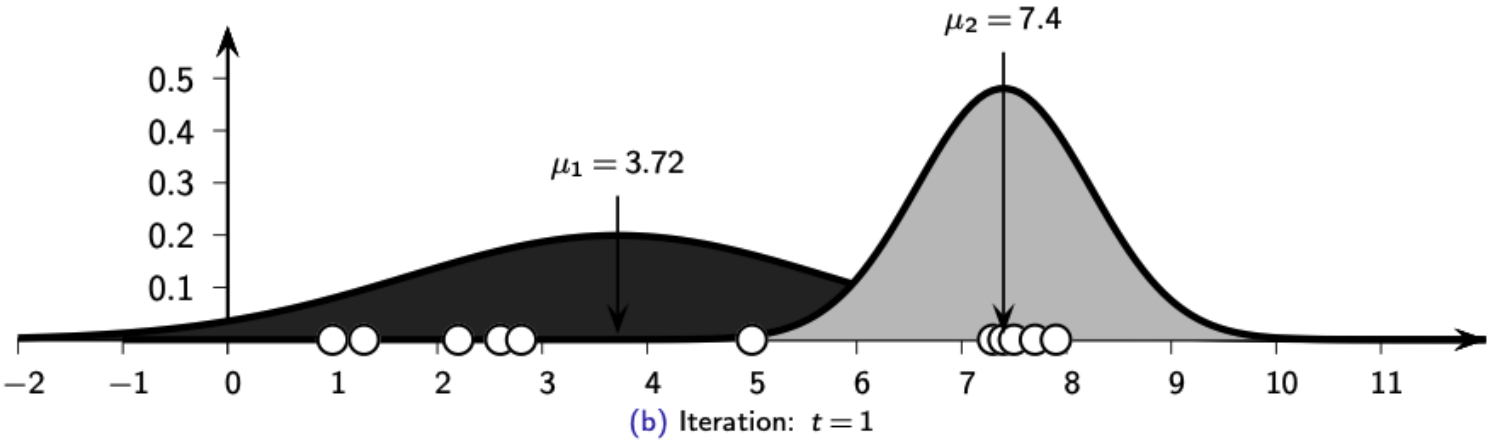
Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data



(a) Initialization: $t = 0$

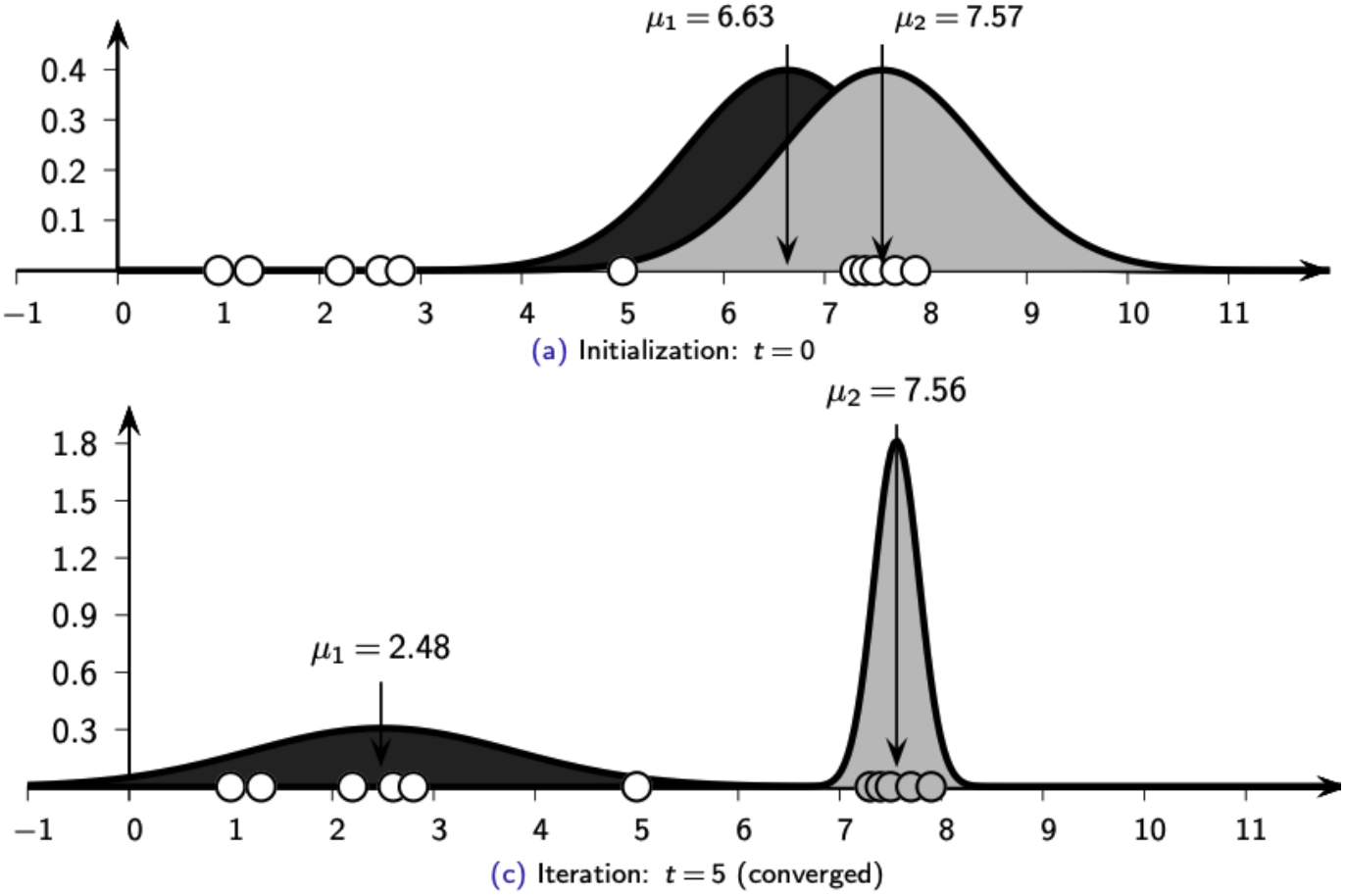


(b) Iteration: $t = 1$

Example: GMM in one dimension

EM algorithm:

Find the best fit models to maximize the likelihood of the data

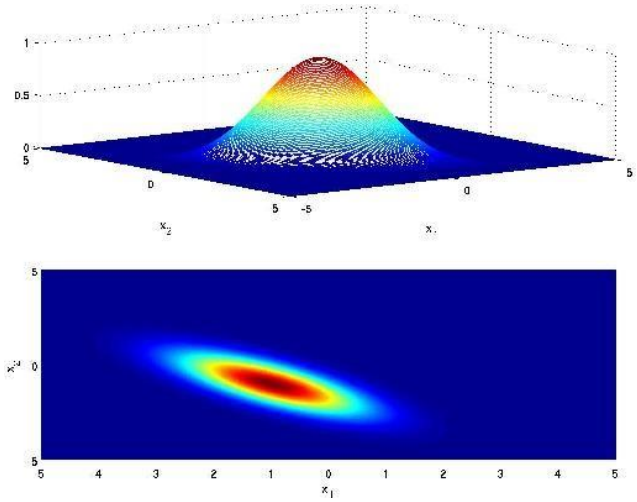


GMM in d-dimensions

- Multivariate Gaussian

$$P(x \mid \mu, \Sigma) = \left|2\pi\Sigma\right|^{-1/2} \exp\left\{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right\}$$

  - x is now a vector, $x \in R^d$
  - μ is the mean vector
  - Σ is the covariance matrix (d x d)



GMM in d-dimensions

- Initialization: $\mu_i, \sum_i$, and $p(C_i)$
- E-step:

$$p_{ij} = p(C_i|x_j) = \frac{p(x_j|C_i)p(C_i)}{\sum_k p(x_j|C_k)p(C_k)}$$

- M-step:
  - Mean:

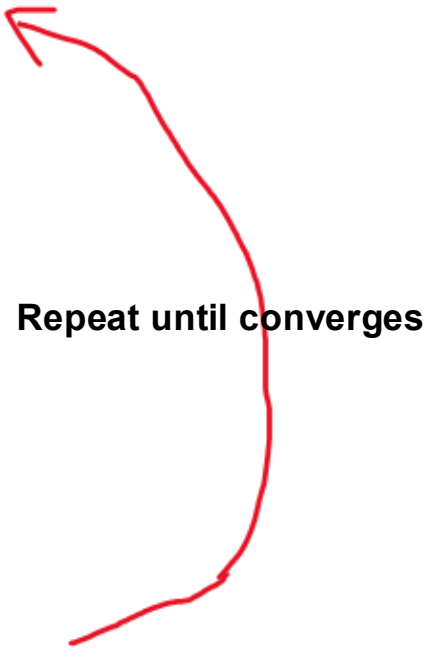
$$\mu_i = \frac{\sum_j p_{ij} x_j}{\sum_j p_{ij}}$$

  - Covariance:

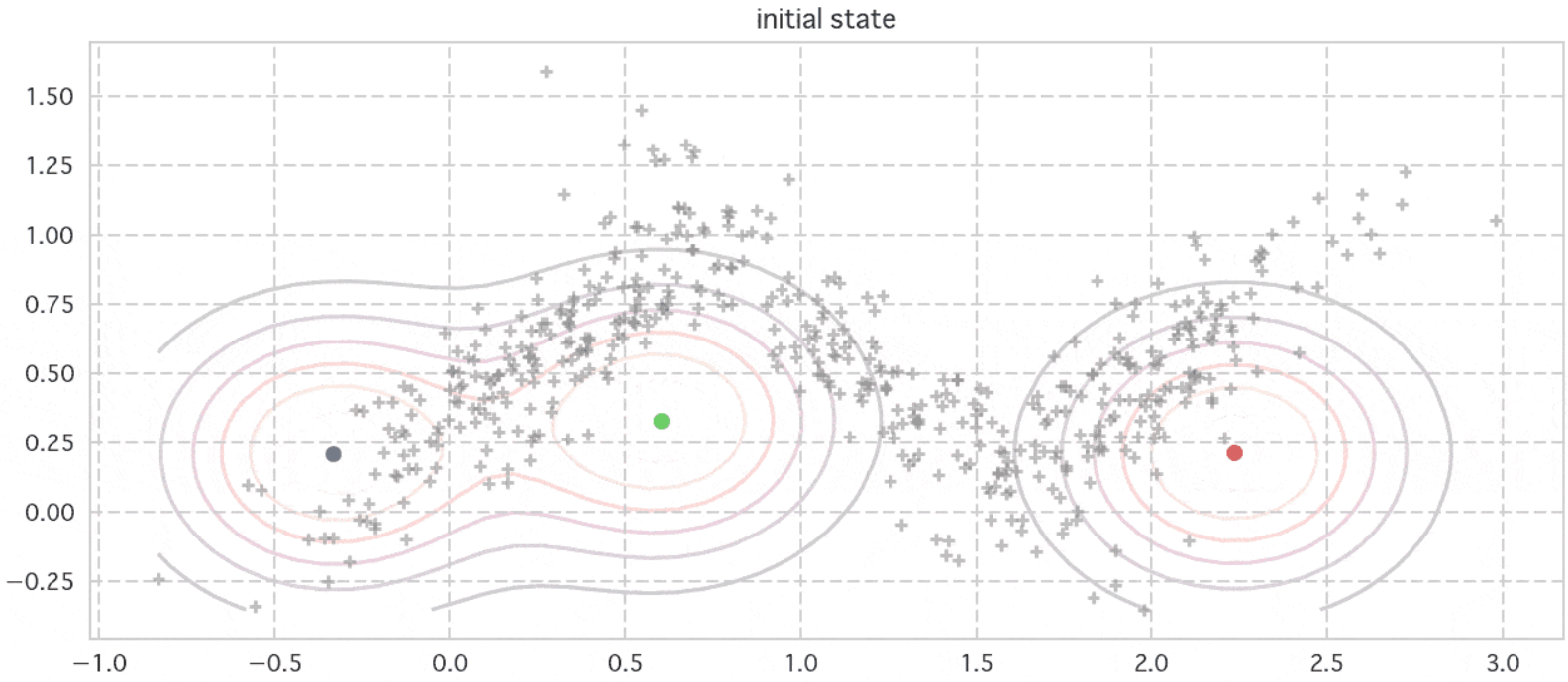
$$\sum_i = \frac{\sum_j p_{ij}(x_j - \mu_j)(x_j - \mu_j)^T}{\sum_j p_{ij}}$$

  - Prior probability:

$$p(C_i) = \frac{\sum_j p_{ij}}{n}$$



Example


initial state
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0

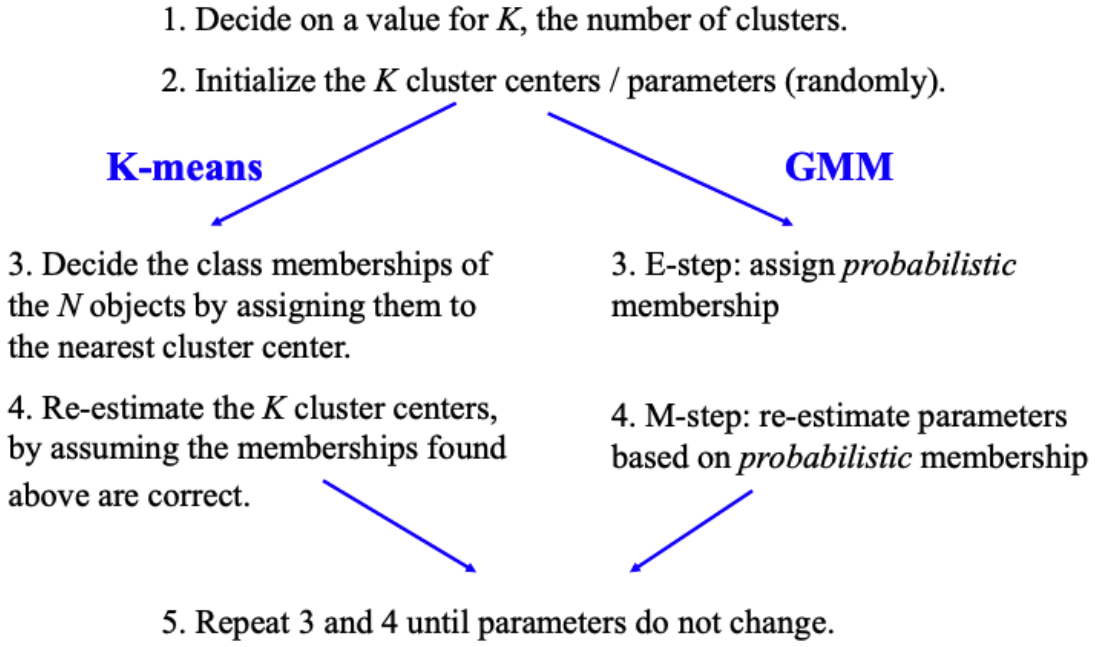
GMM: relation to K-means

K-means as a special case of GMM:

- E-step in GMM: a soft version of K-means.

$$p_{ki} \in [0, 1] \text{ instead of } \{0, 1\}$$

- M-step in GMM estimates the probabilities and the covariance matrix of each cluster in addition to the means.



Summary of GMM

- Strength:
  - Interpretability: learns a generative model of each cluster – you can generate new data based on the learned model
  - Relatively efficient: O(tkn), where n is # objects, k is # clusters, and t is # iterations. Normally, k, t << n.
  - Intuitive objective function: optimizes data likelihood
- Weakness:
  - Often terminates at a local optimum. Initialization is important.
  - Need to specify K, the number of clusters, in advance
  - Not suitable to discover clusters with non-convex shapes
- Summary:
  - GMMs are a ”soft” clustering algorithm, that can be learned using EM.
  - If you keep iterating EM, you will converge, but only to a local optimum.
  - You will see EM in other contexts as well, i.e., Hidden Markov Models

